

视频课程笔记

OpenAI's Yann Dubois: Why AI Progress Suddenly Feels Real

整理: Codex



频道: The MAD Podcast with Matt Turck

发布日期: 2026-05-21

视频时长: 1:14:03

视频链接: <https://www.youtube.com/watch?v=DhD1zZ8w8Mw>

目录

1	为什么进步突然有了体感：连续能力与可靠性阈值	3
1.1	冷启动里的三条线索：可靠性、真实用户、最后一公里	3
1.2	连续能力如何跨过体感阈值	4
1.3	Agent 可靠性的错误累积模型	5
1.4	GPT-5.5 发布中的内部摆动：兴奋、怀疑与外部验证	6
1.5	本章小结	7
2	从纵向能力到横向能力：模型如何被组织成产品	7
2.1	纵向团队：围绕具体场景推进能力	7
2.2	横向团队：指令遵循、函数调用、思考时间与平滑函数	8
2.3	效率曲线：从思考 token 到真实延迟	9
2.4	Yann Dubois 的路径：从 word2vec 到后训练前沿	11
2.5	本章小结	12
3	推理模型的第二阶段：从可验证奖励到真实效用	12
3.1	2026 年的推理：处理含糊任务与真实约束	12
3.2	从数学竞赛和代码题到用户效用	13
3.3	Thinking 与 Pro：同一模型系统上的测试时计算预算	13
3.4	专家式推理：少走岔路、及时回溯、延长有效推理曲线	14
3.5	本章小结	15
4	基础模型管线：预训练、数据墙、多模态与世界模型	16
4.1	预训练仍在扩展：更大的模型如何减少显式思考	16
4.2	数据墙的真实含义：互联网数据、合成数据与高价值数据	17
4.3	多模态与具身智能：常识从哪里来	19
4.4	世界模型与模拟环境：什么时候该停止优化代理目标	19
4.5	本章小结	20
5	后训练的核心机制：SFT、行为克隆与强化学习	20
5.1	中训练：把高质量数据的权重调高	20
5.2	后训练：把会很多知识的模型变成可交互专家	21
5.3	SFT 与行为克隆：从人类答案中复制行为	22
5.4	强化学习：从奖励函数中超过示范答案	22
5.5	RL 是否创造新能力：预训练潜能与后训练显化	24
5.6	本章小结	25
6	扩展 RL 的工程现实：采样、归因、GRPO 与工艺科学化	25
6.1	为什么 RL 曾经显得难以扩展	25
6.2	好先验让 RL 变得可用：从 LLM 到机器人	26
6.3	采样成本与长 rollout 的信用分配问题	26

6.4	GRPO、PPO、DPO：可扩展方法为何胜出	28
6.5	从手艺到科学：复杂系统中的研究节奏	29
6.6	本章小结	30
7	跨领域泛化与幻觉控制：从 GDPval 到负迁移	30
7.1	能力如何跨领域扩散：算法、数据与关注者数量	30
7.2	横向能力的泛化：编辑、指令遵循与多语言迁移	32
7.3	SFT 为什么会放大幻觉，RL 为什么能惩罚编造	33
7.4	负迁移：显式指令遵循与隐式意图理解的冲突	34
7.5	法律、医疗和经济任务为什么仍可训练	35
7.6	本章小结	35
8	评测成为瓶颈：开放任务、专家稀缺与 Model as Judge	35
8.1	模型越强，评测越难	35
8.2	开放任务为什么没有唯一标准答案	37
8.3	人类专家稀缺：当模型超过多数评审者	37
8.4	Model as Judge：用模型评测模型的机会与风险	38
8.5	本章小结	39
9	未来图景：持续学习、Agent Harness 与最后一公里	39
9.1	连续进步与局部跳跃：为什么编码先有体感	39
9.2	持续学习：时间-效用曲线与企业记忆	40
9.3	Agent Harness：短期增益、长期重调与领域适配	42
9.4	最后一公里：权限、连接器、流程嵌入与创业空间	43
9.5	本章小结	44
10	总结与延展：后训练时代的能力、可靠性与产品边界	44
10.1	全片主线压缩：能力、可靠性、评测、产品	44
10.2	从公式到直觉：四条曲线看懂这次进步	45
10.3	对研究者、产品团队和创业者的具体启示	47
10.4	开放问题：持续学习、隐私边界与真实世界奖励	48
10.5	本章小结	48

1 为什么进步突然有了体感：连续能力与可靠性阈值

1.1 冷启动里的三条线索：可靠性、真实用户、最后一公里

开场几分钟里，Yann Dubois 给出的判断很直接：最近几个月的 AI 进步让人有了“突然跨了一档”的体感，核心原因并非某一天凭空长出全新能力，而是几条长期上升的曲线同时越过了用户能感知的线。冷启动里至少有三条线索。

第一条是可靠性。模型以前也能完成片段式任务，能写一段代码、回答一道题、解释一个概念；问题出在持续工作。只要任务拉长，微小错误就会像雪崩前的小裂缝一样累积。Yann 说 OpenAI 内部大约在上一年 12 月感到跨过了一个可靠性门槛：模型开始可以被信任去承担大量实际工作。这里的“信任”很务实，它指的是研究员、工程师和产品团队愿意把一段连续工作交给模型，随后把注意力放在检查关键节点，而非每一步都盯着模型有没有偏航。

第二条是真实用户。视频里反复出现一组转移：从竞赛，到有用，再到用户。数学题、编程比赛和标准化评测提供了可验证奖励；真实世界里的编码、知识工作、电脑使用和早期科学研究提出了更粗糙、更开放的目标。用户在意的结果也更尖锐：能不能节省时间，能不能接住上下文，能不能在工具链里完成闭环。

第三条是最后一公里。模型能力需要进入具体工作流，才会变成产品体验。所谓最后一公里，指权限、工具、上下文、接口、组织流程、行业语境这些看似琐碎的东西。它像桥梁的引道：主桥再壮观，车开不上去，交通价值也会大打折扣。Yann 在开场就强调，不同垂直领域里仍会留下大量最后一公里空间，这也是后文谈组织结构、横向能力和 Agent Harness 的伏笔。



图 1: Yann Dubois 在解释近期 AI 进步为何产生阶跃体感时的访谈画面；这一帧用于给可靠性阈值讨论建立视频来源锚点。¹

¹视频帧来源：00:03:35，属于 Section 1 的 00:01:30–00:07:33 取样区间。

可靠性阈值

可靠性阈值指 Agent 型模型在持续、多步骤、有工具参与的工作中，达到用户愿意委托任务的最低稳定水平。它不等同于单题准确率；它关心的是长链条里灾难性错误是否低到足以让人把模型当作协作者。

这里有一个容易被低估的概念差异：单点能力像短跑成绩，可靠性像长途运输的准点率。短跑成绩高说明爆发力强；长途运输还要看每个中转站、每次装卸、每段路况。Agent 型模型也是如此。它会读文件、写代码、调用工具、等待结果、再决定下一步。任何一步都可能让后续路径偏离。

1.2 连续能力如何跨过体感阈值

Yann 的关键解释是：底层能力进步相当连续，用户体感却可能呈现阶跃。这个现象并不神秘。许多工程系统都有类似结构：水位每天涨一点，堤坝溢流发生在某个瞬间；网络带宽逐年增加，高清视频体验在缓冲消失的那一刻才被用户记住；电池密度缓慢提升，电动车进入日常通勤半径后才显得“突然可用”。

AI 模型也有这种门槛效应。只会写函数片段时，它像一个聪明但需要持续照看的实习生；能连续定位问题、改代码、跑测试、解释失败、再修复时，它开始像一个可以接活的工程搭档。能力曲线的斜率也许平滑，产品体验的评分函数却带有强阈值：没过线时，人类的监督成本吃掉收益；过线之后，监督变成抽查和验收，净收益迅速显现。

这种体感在编码任务上更早出现，原因很现实。编码有大量中间反馈：类型检查、单元测试、运行日志、CI 结果、编译错误、benchmark。模型做错时，环境常常能快速给出负反馈。相比之下，法律意见、医疗建议、战略判断等任务的反馈更稀疏、更慢、更依赖专家。于是同样的模型进步，会先在反馈密集领域形成局部跳变。

Small per-step error reductions compound in long agent tasks

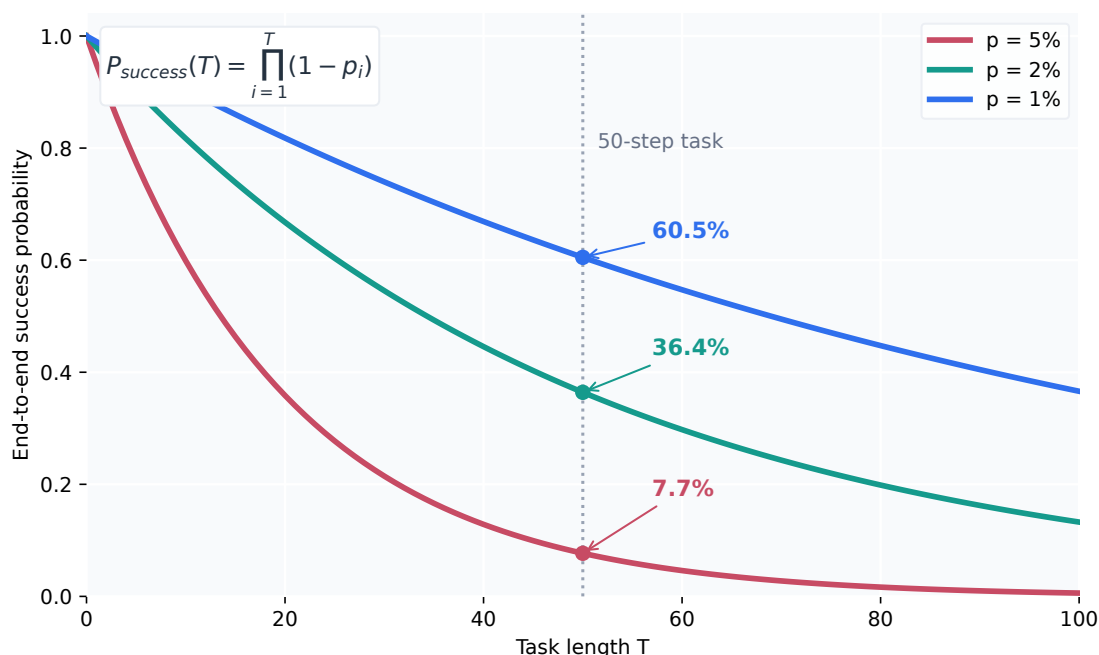


图 2: 多步 Agent 任务会把单步错误率放大成整体成功率差异。50 步任务中, 单步错误率从 5% 降到 2%, 整体成功率从约 7.7% 升到约 36.4%。

不要把阶跃体感误读成神秘涌现

本章说的阶跃体感, 是连续能力穿过产品阈值后的用户感知变化。它不要求能力曲线在数学上突然断裂, 也不自动证明模型内部出现了单一、可定位的新模块。

这条线索也解释了为什么内部使用者会比外部用户更早感到变化。OpenAI 内部研究员和工程师既是模型构建者, 也是高频用户。他们把模型用于训练其他模型、搭建研究工具、加速代码工作。模型一旦越过可靠性阈值, 就会反过来加速研究和工程本身, 形成“工具改善生产工具”的循环。这个循环不会让每一天都像发布会一样惊人, 却会让几个月后的系统能力看起来明显换挡。

1.3 Agent 可靠性的错误累积模型

Yann 在解释可靠性时用了—个非常朴素的说法: 可以把 Agent 型模型想象成每两分钟都有一定概率犯错; 它运行得越久, 最终答案出错的概率越高。这个直觉可以写成一个简单模型。假设一个任务被拆成 T 个连续工作区间, 每个区间出现关键错误的概率为 p_i 。如果每一步都必须不出关键错误, 整体成功概率可以近似写为:

$$P_{\text{success}}(T) = \prod_{i=1}^T (1 - p_i) \approx \exp\left(-\sum_{i=1}^T p_i\right).$$

符号解释:

- T : 任务包含的工作区间数量。视频里用“每两分钟”做直觉说明, 实际系统中可以按一次工具调用、一次代码编辑、一次推理段落或一次检查点划分。

- p_i : 第 i 个区间发生关键错误的概率。关键错误指足以影响最终结果的偏航，例如误读需求、调用错工具、改坏文件、忽略测试失败。
- $1 - p_i$: 第 i 个区间不发生关键错误的概率。
- $P_{\text{success}}(T)$: 整个多步任务在没有关键错误的情况下完成的概率。
- $\prod_{i=1}^T$: 把所有步骤的成功概率相乘，表达“每一步都要过关”的链式结构。
- $\exp\left(-\sum_{i=1}^T p_i\right)$: 当每一步错误概率较小时的近似，强调总风险大致随错误概率之和增长。

这个公式的教学价值在于，它让“可靠性小幅提升为什么有巨大体感”变得可见。若每一步关键错误率从 5% 降到 2%，单步看只是 3 个百分点；放进 50 步任务，差异会被放大。前者的成功概率约为 $0.95^{50} \approx 7.7\%$ ，后者约为 $0.98^{50} \approx 36.4\%$ 。用户不会说“模型提升了 3 个百分点”，用户会说“以前跑不完，现在经常能跑完”。

这个模型也提醒读者：Agent 可靠性包含模型侧和应用侧两部分。Yann 在回答中刻意区分了两者。他说应用团队也做了大量可靠性工作，但自己当时强调的是模型本身的错误概率下降。应用侧可以通过权限隔离、工具校验、回滚、测试、人工确认来降低错误的后果；模型侧降低的是 p_i 本身。一个生产级 Agent 需要两边同时推进，因为长任务里任何单独一边都很难扛住全部风险。

读者可能忽略的一层：错误并不平均

公式把每个区间写成 p_i ，而没有写成统一的 p ，是为了保留任务结构。真实 Agent 工作里，需求理解、文件改写、外部工具调用、最终汇报的风险不同；一个早期理解错误会污染后面所有步骤，尾部表达错误则可能只影响交付说明。

可靠性阈值因此也不是一个全球统一数值。写测试覆盖良好的后端函数，阈值可以较低，因为环境会及时反馈；操作生产数据库、写医疗建议、处理法律风险，阈值必须更高，因为错误代价和验证成本都更大。产品团队真正要问的是：在这个任务分布、这个工具边界、这个人工验收流程下， P_{success} 是否高到足以覆盖监督成本和错误成本。

1.4 GPT-5.5 发布中的内部摆动：兴奋、怀疑与外部验证

视频里谈到 GPT-5.5 时，Yann 给了一个很有现场感的内部过程。公司里很多人对这个模型非常兴奋，因为它在 Agent 编码、电脑使用、知识工作和早期科学研究等方向表现突出，也因为许多团队都参与了构建。随后，怀疑开始出现：一个早期看起来很强的模型，总会暴露出一些不稳、偏科或意外失败的地方。内部情绪会出现一波波上升和下降，直到发布后被外部用户实际检验。

Yann 还用了两个很有画面感的说法。第一，GPT-5.5 像是“all the stars were aligned”，许多团队的贡献在同一窗口对齐；第二，它的内部情绪波动幅度更高，早期兴奋、随后怀疑、再到发布后的外部验证，比普通模型更像一条高振幅曲线。这能帮助读者理解，前沿模型发布是一组不稳定信号最终被产品窗口压缩到一起。

这段描述很重要，因为它让读者看到前沿模型发布的真实状态：强模型通常伴随不确定性。内部兴奋来自局部样例、早期评测和使用体感；内部怀疑来自更广的任务分布、失败案例和产品风险。两者都合理。没有兴奋，团队可能低估突破；没有怀疑，团队会把漂亮 demo 误当成稳定产品。

发布前情绪不能替代评测

内部兴奋和内部怀疑都是信号，但都带有采样偏差。真正有价值的是把这些信号变成覆盖更广的 eval、真实用户反馈和失效案例分析。否则，团队只是在高光样例和失败样例之间摇摆。

Yann 还提到，越靠近产品的阶段，迭代周期越短；越靠近上游训练，周期越长。预训练、中训练、后训练和产品推送处在不同时间尺度上：上游可能以月计，后端集成和产品侧可能以天计。这会加剧发布前的摆动。一个上游决定的影响很大却反馈慢，一个下游修复反馈快却受上游能力约束。当前沿模型接近发布窗口，所有时间尺度被压缩到同一个发布节奏里，组织的兴奋和怀疑都会被放大。

因此，GPT-5.5 在视频中的意义不只是一个型号名称。它被用来说明一个系统性现象：连续的模型能力、后训练进展、真实用户任务、推理效率和组织协同在某个时间点合流，于是体感出现跳变。这个“突然”有技术原因，也有产品原因和组织原因。

1.5 本章小结

本章建立了整篇笔记的第一条主线：AI 进步的体感跳变，来自连续能力跨过可靠性阈值。冷启动里的三条线索分别是可靠性、真实用户效用和最后一公里。可靠性尤其关键，因为 Agent 型模型面对的是多步链式任务，单步错误会累积成整体失败。

用公式看， $P_{\text{success}}(T) = \prod_{i=1}^T (1 - p_i)$ 把这个问题压缩得很清楚：长任务的成功率对每一步错误概率极其敏感。小幅降低 p_i ，会在长链条里产生大幅体感差异。GPT-5.5 发布前后的兴奋和怀疑，则说明前沿模型产品化并非单次跑分胜利；它是一组能力、评测、用户反馈和组织节奏的共同收敛。

2 从纵向能力到横向能力：模型如何被组织成产品

2.1 纵向团队：围绕具体场景推进能力

上一章讨论了可靠性阈值，接下来要看一个更工程化的问题：一个前沿模型怎样从许多零散能力，变成用户手里感觉统一的产品。Yann 在 00:07:33 之后的回答里给出了组织视角。OpenAI 内部会有不同团队围绕具体用例推进能力，例如 Agent 编码、电脑使用、知识工作、科学研究等。这类团队可以称为纵向团队。

纵向团队的优势在于目标具体。编码团队可以盯住代码库导航、补丁质量、测试反馈、长上下文编辑；电脑使用团队可以盯住屏幕理解、点击动作、浏览器状态和错误恢复；科学研究方向可以关注论文理解、实验设计、数据分析和严谨引用。每条纵向线都像一支钻井队，往某个任务地层深处打。只要反馈足够清晰，它们就能快速发现模型在哪些环节还不够用。

这也解释了为什么同一个模型发布时会被描述为“在若干方向尤其强”。它的进步通常带有任务分布上的侧重：多个纵向任务获得了实打实的改进，平均意义上的整体提升只是最终呈现。用户看到的是一个统一名称；内部实际发生的是许多团队在不同任务分布上试探、训练、评估和集成。

纵向能力/横向能力的产品含义

纵向能力指模型在某个具体场景里的任务完成力，例如编码、电脑使用、科学研究、客服、金融分析。横向能力指跨场景共享的行为质量，例如指令遵循、函数调用、记忆、思考时间控制、拒答边界和格式稳定性。产品化模型需要两者叠加：纵向能力让模型在场景里有用，横向能力让模型在不同场景里像同一个可靠产品。

这里有一个产品经理和研究员都容易低估的事实：纵向能力如果不经整合，会变成“尖刺”。模型在某个场景里很强，在另一个场景里却突然怪异；一个模式下很会调用函数，另一个模式下又不听格式；写代码时很谨慎，回答普通问题时又过度推理。用户不会把这些问题归因到内部团队边界，他们只会觉得产品不稳定。

2.2 横向团队：指令遵循、函数调用、思考时间与平滑函数

Yann 说自己的团队承担两类职责：一类是把纵向改进放进最终模型，另一类是做横向改进。他用了一个很好的说法：团队像一个 smoothing function，也就是平滑函数。这个比喻的意思是，许多局部改进进入一个最终模型时，需要被调和成一致、连贯、不过分尖刺的行为。

他对 Post-training Frontiers 团队的描述还包含三项很操作化的职责。第一，决定哪些来自纵向团队的改进能进入最终 run；第二，给其他团队提供接近最终 run 的 science experiments，让他们能在代表性环境里迭代；第三，把所有改进真正合到一次大训练里，这既需要 GPU 基础设施，也需要机器学习层面的整合判断。只有在这三件事之上，horizontal improvements 才能稳定落到产品模型里。

平滑函数并不意味着抹平所有差异。好的产品模型应该在编码时像工程助手，在写作时像编辑，在数据分析时像分析师；问题在于这些角色不能互相打架。指令遵循、函数调用、思考时间、记忆和通用行为质量，就是横向团队要照看的层面。

指令遵循负责把用户的显式要求稳定转成行动约束。它听起来基础，却决定了产品可信度：用户要求中文，模型不能突然切英文；用户要求只输出文件路径，模型不该附赠长篇解释；用户要求不要改别人的文件，模型必须把这个约束贯穿全程。

函数调用负责把语言意图映射到工具接口。模型如果说“我需要查一下”，却不能正确填参数、处理错误和解释返回值，工具能力就停在口头层面。函数调用能力像手眼协调：大脑有想法，手要能准确拿起工具。

思考时间负责决定模型在不同问题上花多少计算。简单问题过度思考会浪费延迟，复杂问题思考不足会损害正确性。这个控制项非常横向，因为它影响所有任务：写代码、做数学、处理表格、规划行程、总结文档都需要在“快”和“准”之间调度。

From vertical capabilities to one product model

Vertical teams push concrete use cases; horizontal teams stabilize cross-domain behavior.

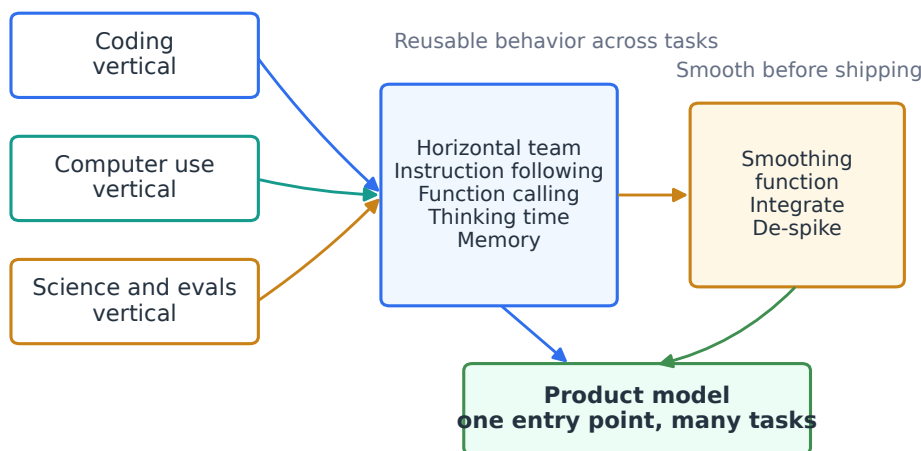


图 3: 纵向团队深入具体任务分布, 横向团队沉淀指令遵循、函数调用、思考时间和记忆等跨域能力, 平滑函数把这些改进整合进同一个产品模型。

平滑函数的组织含义

smoothing function 指把多个纵向团队带来的局部能力提升, 整合成一个行为连续、风格稳定、跨场景不尖刺的最终模型。它关注的核心是用户在不同任务之间切换时的整体一致性, 单一 benchmark 的最高点只能提供局部证据。

这种组织方式还有一个重要好处: 纵向改进和横向改进可以相对正交地推进。某个版本里, 部分纵向团队成果进入最终 run; 下个版本, 另一些纵向团队成果再进入。横向团队则持续维护底座行为, 使模型不会因为某个方向的局部优化而牺牲其他方向的可用性。

2.3 效率曲线: 从思考 token 到真实延迟

视频里谈到 GPT-5.5 时, Yann 特别强调效率。他说许多任务现在可以大约以两倍速度完成。这个“速度”不是单一工程优化的结果; 它来自研究、推理服务和产品目标的共同压缩。

研究侧看的第一条曲线, 是测试时计算曲线。横轴可以先理解为模型在回答时消耗的思考 token 数, 也就是 k ; 纵轴是表现、正确率或任务效用。研究目标是把曲线向左移动: 用更少思考达到同样表现, 或者在同样计算下得到更好结果。

推理服务侧则把这个 k 映射成用户可感知的延迟。最终用户不直接感受 token 数; 用户感受的是等待几秒、几十秒, 还是几分钟。可以用一个抽象公式表示:

$$L = g(k, M, H, B, R).$$

符号解释:

- L : 用户可见延迟, 单位可以是秒或分钟。

- $g(\cdot)$: 服务系统的映射函数，把测试时计算转换为真实等待时间。
- k : 测试时计算预算，可粗略理解为思考 token、工具调用轮次或中间推理开销。
- M : 模型本身的规模、结构和推理特性。不同模型在同样 k 下的运行成本不同。
- H : 硬件和推理基础设施，包括 GPU/加速器、并行策略、缓存和调度。
- B : 批处理和流量状态。高峰期、批大小、队列策略都会改变同一请求的等待时间。
- R : 路由和产品策略，例如是否启用更强模式、是否调用外部工具、是否做额外检查。

这个公式的作用是把“效率”从口号变成系统问题。若只减少 k ，模型可能更快却更粗糙；若只优化 H ，服务更快但模型本身仍然了嗦；若路由 R 设计得不好，简单问题可能被送进昂贵路径，复杂问题却被过早截断。产品里的真实效率，是研究曲线和服务曲线叠在一起后的结果。

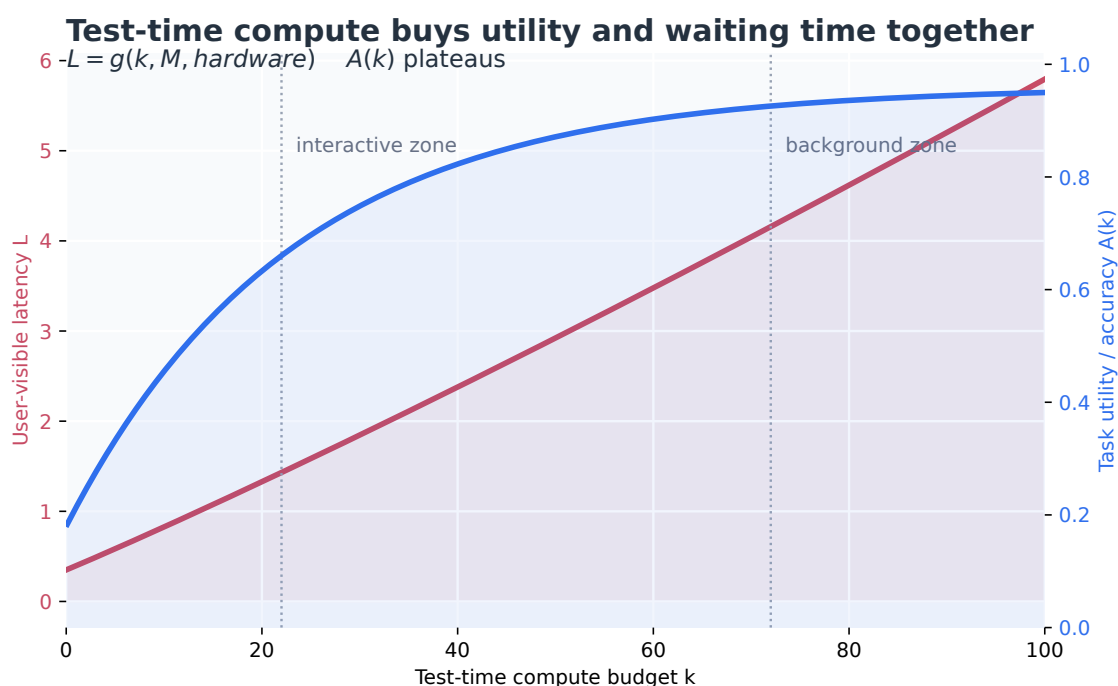


图 4: 测试时计算预算 k 同时影响效用 $A(k)$ 和用户可感知延迟 L 。交互任务需要低等待，高价值后台任务可以把预算推向更长推理。

延迟不是 token 数的简单同义词

思考 token 是研究侧的近似横轴，延迟是用户侧的真实横轴。两者相关，但中间隔着模型结构、硬件、批处理、调度、工具调用和产品路由。把二者混为一谈，会误判用户体验。

用一个类比看， k 像汽车发动机实际做功， L 像从家到公司的通勤时间。发动机效率重要，道路、红绿灯、导航路线和拥堵也重要。模型效率同理：后训练让模型“少想也能想对”，推理系统让每个 token 更快，产品路由让计算花在值得花的请求上。

2.4 Yann Dubois 的路径：从 word2vec 到后训练前沿

Yann 的个人路径在这一节里不是传记装饰，它解释了为什么他会用“离散到连续”“语义空间”“多语言”和“后训练”这些视角看模型产品化。他本科在瑞士读生物医学工程，后来在加拿大交换时接触到 word2vec。这个算法把离散词语放进连续向量空间：语义相近的词在空间中距离更近。对许多人来说，word2vec 是一次概念启蒙，因为它让“语言意义”第一次像几何对象一样可以被计算。

word2vec 的直觉

word2vec 把词表示成向量。若用 $\mathbf{v}_{\text{king}}$ 、 $\mathbf{v}_{\text{queen}}$ 表示词向量，经典直觉是语义关系可以近似表现为向量方向。它的历史意义不在于解决了语言理解，而在于让离散符号进入可优化的连续空间，为后来的深度 NLP 和 Transformer 时代铺路。

2017 年前后，Yann 曾以为英语 NLP 接近解决，于是转向数据较少的语言。他在新加坡 Grab 做过 Khmer、Bahasa、Thai、Vietnamese 等语言的自然语言处理管线。这个经历很值得放进本章，因为多语言工作会迫使研究者面对数据分布、资源不均、任务迁移和产品落地。它天然连接纵向和横向：每种语言都有本地需求，但底层语言能力和指令能力又需要共享。

随后他在 Stanford 做博士，并参与 Stanford Alpaca。Alpaca 在现代后训练社区里有标志性意义：它让更多研究者看到，用指令数据和较轻量的训练流程，可以把基础模型改造成更会对话、更会遵循任务的模型。这个历史节点把前训练时代的“模型会很多东西”，推进到后训练时代的“模型能以用户需要的方式做事”。

Previously, I was a PhD student at Stanford University, co-advised by **Percy Liang** and **Tatsu Hashimoto** and funded by a **Knight-Hennessy Scholarship**. My research focused on instruction following (**Alpaca**, **Alpaca-eval**). Before that, I traveled around doing research in representation learning and worked at startups including **Grab**, where I built NLP systems for under-researched languages (Thai, Khmer, Burmese, ...) before LLMs were a thing.

Note: please do not contact me about quant jobs; I'm not interested regardless of compensation.

图 5: Yann Dubois 个人经历页中可读的 Alpaca 与 Alpaca-eval 线索；它把 Section 2 的组织 and 效率讨论连接到后训练与指令遵循研究背景。²

进入 OpenAI 后，Yann 领导 Post-training Frontiers 相关团队，工作重心也延续了这条线：把模型潜在能力整理成可用行为，把不同团队的改进放进最终 run，把横向质量做稳。这里的“前沿”不是只追求更高参数量；它还包括让模型在真实产品里更一致、更高效、更会调用工具、更懂

²视频帧来源：00:14:20，属于 Section 2 的 00:12:32–00:15:37 取样区间。

什么时候该多想。

两个小插曲能补足人物语气：他曾在个人页面上提醒量化基金不要联系自己，因为他更关心工作的正向影响；GPT-5 发布视频里的法语教学 app 是真正 live 的，最后一次 rehearsal 还失败过，所以这个 demo 本身也反映了“可靠性刚过阈值”时的紧张感。

2.5 本章小结

本章把模型产品化拆成两类组织能力：纵向团队深挖具体场景，横向团队维护跨场景行为。纵向能力带来局部突破，横向能力负责指令遵循、函数调用、记忆、思考时间和整体一致性。Yann 所说的 smoothing function，正是把多个局部改进整合成一个稳定产品的组织机制。

效率部分说明，测试时计算 k 需要通过 $L = g(k, M, H, B, R)$ 才变成用户感知的延迟。研究把性能曲线左移，推理系统缩短等待，产品路由决定计算花在哪里。Yann 从 word2vec、多语言 NLP、Stanford Alpaca 到 OpenAI 后训练前沿的路径，也说明了同一条主线：语言模型的价值不止在会什么，还在它能否被组织成稳定、可控、低延迟的用户产品。

3 推理模型的第二阶段：从可验证奖励到真实效用

3.1 2026 年的推理：处理含糊任务与真实约束

视频在 00:15:37 之后把问题从“模型能不能想更久”推进到“模型想更久之后，能不能在真实任务里更有用”。Yann Dubois 对 2026 年推理模型的判断很清楚：o1 和 o1-preview 证明了一个关键现象，模型花更多时间思考，正确率会提高；但早期证据主要来自数学、代码竞赛等容易判对错的任务。到了 2026 年，真正的变化是这套推理工具被带进更含糊、更开放、更接近用户工作的场景。

这里的“含糊”并不等于没有标准。真实任务里也有好坏之分，只是标准通常分散在上下文、约束、用户意图、工具状态和交付成本里。一个模型帮用户整理混乱表格、修复真实代码库、解释研究材料、规划实验或做经济任务分析时，答案往往没有单一的标准标签。用户关心的是能不能节省时间、减少返工、发现关键错误、把工作推进到可验收状态。

可验证奖励
可验证奖励指可以自动或近似自动检查正确性的奖励信号。数学题、代码竞赛、单元测试、形式化证明片段、部分安全挑战，都属于这种范畴。它的优势是反馈清楚：输出正确得分高，输出错误得分低；它的局限是覆盖范围窄，容易把训练目标锁在“可判题”的任务分布上。
真实效用
真实效用指模型在开放任务中为用户带来的实际价值，包括节省时间、提高决策质量、减少操作成本、降低返工概率、让复杂工作进入可交付状态。它经常无法由一个简单标签决定，需要把任务上下文、用户偏好、领域约束和最终工作流一起纳入判断。

这两个定义解释了 Yann 所说的“第二阶段”。第一阶段证明推理模型在可验证奖励上可训练：给出题目、让模型采样推理、用明确判据打分，再强化高分路径。第二阶段要面对真实效用：用户

任务更像一片雾中的地形，模型既要找路，也要判断哪条路对用户有价值。可验证奖励像棋盘上的落子规则，真实效用更像带着时间、预算、风险和偏好的项目交付。

3.2 从数学竞赛和代码题到用户效用

早期推理模型偏爱数学和代码竞赛，有技术上的合理性。数学题天然有答案，代码题可以运行测试，竞赛平台提供大量结构化题目和清晰反馈。训练系统很容易知道“这条推理链最后成功了吗”。这类任务让研究者验证了测试时计算（test-time compute）的基本事实：模型花更多计算尝试、检查、推导，整体正确率会上升。

但真实用户任务不会总是长成竞赛题。Yann 在 00:17:04–00:18:37 的解释里强调，过去一年到 2026 年初，团队把“有可验证奖励时有有效的论证和方法”迁移到真实世界，目标变成优化用户效用和生产力。他还提到，当前 eval 看起来比早期 Codeforces 一类任务更现实，例如 GDPval、SWE-bench 或 SWE-bench Pro 这类更贴近经济工作和真实代码维护的评测。

这种转向带来一个训练哲学变化：奖励信号从“答案是否等于标准答案”扩展为“产出是否帮助用户完成任务”。在代码库修复中，测试通过只是最低门槛；补丁是否小、是否符合项目风格、是否避免破坏其他模块，也都影响效用。在研究工作中，答案是否引用了正确证据、是否避开过度外推、是否把不确定性讲清楚，比单句结论更重要。

把真实效用简化成主观偏好会误导训练

真实效用并非随意打分。它需要更贴近工作流的评测设计：输入要像真实任务，工具环境要能暴露失败，验收标准要覆盖质量、成本、风险和上下文一致性。若只把它当作“人喜欢什么答案”，模型会学到讨好式表达，却未必学到可靠交付。

这也是推理模型必须继续结合后训练和强化学习的原因。只会长篇思考的模型可能在错误方向上花更多成本；真正有用的模型要能判断哪些分支值得展开，哪些假设该早停，哪些工具调用能最快缩小不确定性。用户购买的核心价值是更高概率的正确交付，“思考痕迹”只是中间过程。

3.3 Thinking 与 Pro：同一模型系统上的测试时计算预算

在 00:18:34–00:20:09，主持人问 GPT-5.5 Thinking 和 GPT-5.5 Pro 的区别是否只是更多测试时计算、更多 token 和更多时间。Yann 的回答基本是肯定的：关键在于向模型以及整个交付系统投入多少测试时计算。这里的 GPT-5.5 Thinking 和 GPT-5.5 Pro 应当按视频语境理解为产品模式名称；本文不把它们外推为任何未被材料支持的发布细节。

为了把这个差异说清，可以把测试时计算预算记为 k ，把性能、正确率或任务效用记为 $A(k)$ 。最朴素的直觉是 k 增大，模型有更多机会搜索、验证、回溯， $A(k)$ 随之上升。但 Yann 也强调，曲线有明显平台效应，收益不会线性增长。一个常用的示意公式是：

$$A(k) = A_{\max} - c(k + k_0)^{-\alpha}, \quad c > 0, k_0 > 0, \alpha > 0.$$

符号解释：

- k ：测试时计算预算，可粗略理解为思考 token、搜索分支、工具调用轮次或更长推理时间的综合预算。

- $A(k)$: 在预算 k 下达到的性能、正确率或用户效用。
- $A_{\max}$: 在当前模型和任务分布下可接近的性能上限。
- c : 距离上限的尺度因子，表示低预算时还有多少可改进空间。
- k_0 : 基础预算项，避免在 $k = 0$ 时公式退化，也表示系统已有的固定推理开销。
- α : 收益递减速度； α 越大，增加计算带来的早期收益越明显，后期越快接近平台。

这个公式表达的是一条“越想越好，但越往后越贵”的曲线。Thinking 可以理解为在较常用的 k 区间运行：用户还需要交互速度，等待成本不能过高。Pro 则把预算推到更右侧：它允许模型运行更久，换取更高的正确概率。Yann 提到自己不常用 Pro，因为他不喜欢等；但有些数学家或学术研究者很喜欢 Pro，因为他们可以让任务在后台跑一两个小时，交互速度的权重较低。

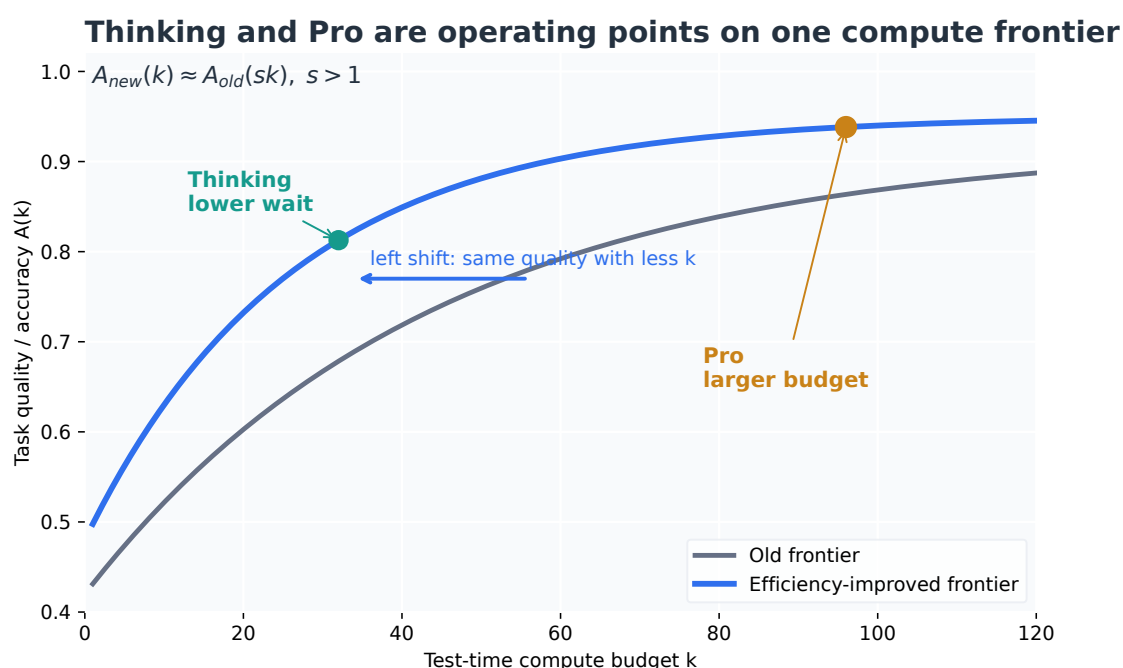


图 6: Thinking 与 Pro 可以理解为同一测试时计算前沿上的不同工作点：Thinking 选择较低等待，Pro 用更大预算换取更高正确率；效率提升则表现为曲线左移。

这条曲线还解释了“预算”和“模式”的产品含义。若用户正在问一个普通事实、改一段短文字、做一次轻量代码补全，把 k 推大只会制造延迟。若用户在睡前提交一个复杂证明、深度研究任务或长代码迁移，延迟的机会成本下降，最大化正确概率就更合理。产品里的 Thinking/Pro 差异，实际是在同一性能-计算预算曲线上选择不同工作点。

3.4 专家式推理：少走岔路、及时回溯、延长有效推理曲线

主持人在 00:21:34 附近追问：推理变强在实践中到底意味着什么，是否包括模型走错方向后更早中断自己。Yann 用人类专家和本科新手作类比。新手面对陌生问题，会探索许多可能方向，花一两天试错；专家通常能更快判断哪条路径更可能正确，少在低价值分支上浪费时间。这就是推理效率的核心。

从曲线角度看，效率改进有两种效果。第一种是左移：同样的性能用更少预算达到，可以写成

$$A_{\text{new}}(k) \approx A_{\text{old}}(sk), \quad s > 1.$$

符号解释：

- $A_{\text{old}}(k)$ ：旧模型或旧训练流程在预算 k 下的性能曲线。
- $A_{\text{new}}(k)$ ：经过效率改进后的新曲线。
- s ：有效计算放大系数； $s > 1$ 表示同样的实际预算 k ，新模型像获得了更多有效推理预算。

第二种是延长有效曲线：Pro 模式把模型推到更长运行时间，曲线右端仍然能挤出收益。只有当模型学会更好地分配推理努力时，右端延长才有意义。否则模型只是把更多时间花在重复、绕圈或错误假设上，用户得到的只是更贵的失败。

专家式推理至少包含三类能力。第一，少走岔路。模型在早期就能识别高概率方向，像经验丰富的工程师看到报错堆栈后先查最可能的配置问题。第二，更快回溯。当中间证据显示当前假设站不住脚，模型能停止沉没成本，改走其他路径。第三，保留长程耐心。复杂任务确实需要时间，模型不能在刚出现困难时就给出草率答案。

有效推理预算

有效推理预算指真正转化为更高任务质量的那部分测试时计算。增加 token、延长等待时间、启动 Pro 模式，都只是在增加名义预算；模型能否少走岔路、及早回溯、把长推理用于关键分支，才决定名义预算有多少变成有效预算。

这解释了为什么“更多计算”和“更高效率”可以同时成立。新一代模型把曲线向左推，使常规任务更快；Pro 又把可用曲线向右延展，使高价值任务有机会用更长计算换取更高正确率。两者服务不同使用场景。日常工作需要低延迟、高吞吐；数学研究、复杂代码迁移、严肃分析则更愿意用等待换把握。

3.5 本章小结

本章把推理模型的第二阶段定义为从可验证奖励走向真实效用。可验证奖励提供了早期推理突破的训练土壤，因为数学和代码竞赛能清楚判对错；真实效用要求模型在含糊任务、真实工具、用户约束和交付质量之间做权衡。2026 年推理模型的关键进展，正是把“越想越可能正确”的能力迁移到更接近真实工作的任务分布。

但推理曲线不能脱离基础模型本身：模型能否少想也想对，仍取决于预训练获得的先验、数据配方和多模态世界知识。下一章回到这条更底层的管线。

Thinking 和 Pro 可以用性能-计算预算曲线 $A(k) = A_{\text{max}} - c(k + k_0)^{-\alpha}$ 理解：Thinking 选择较低延迟的常用预算区间，Pro 把预算推向右侧，用更长等待换更高正确概率。推理效率的提高体现在曲线左移和有效右端延长上。专家式模型会少走岔路、更早回溯，并把长时间推理花在真正有信息增益的分支上。用户最终感受到的进步，是同样时间里更可靠，或者同样难题上更有把握，而非“模型想了更多字”这个表面量。

4 基础模型管线：预训练、数据墙、多模态与世界模型

4.1 预训练仍在扩展：更大的模型如何减少显式思考

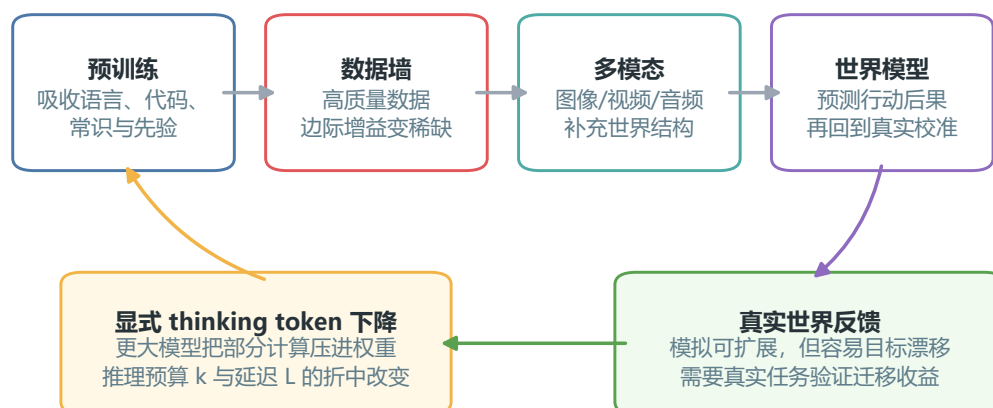
在 00:23:23–00:27:03，话题从后训练和推理转向现代 AI 系统的基础管线：预训练、中训练、后训练。主持人直接提出了一个 2025 年以来常见的判断：预训练似乎碰到了墙，继续扩大可能不再带来明显进步。Yann 的回答很克制。他明确说自己不能详细谈 OpenAI 内部发生了什么，只能说团队做了大量工作，模型还在持续变好。写这一章时也必须保留这个边界：下面讨论的是视频中明确提到的机制和可公开理解的工程直觉，不推测 OpenAI 内部细节。

预训练的基本作用，是在大规模数据上学习语言、世界知识、统计规律、推理先验和各种任务的隐含结构。后训练可以把能力整理成更好用的行为，但如果基础模型没有足够丰富的先验，后训练很难凭空补出大量世界知识。Yann 在这里补充了一个容易被忽略的效率机制：更大的模型往往可以减少显式思考 token，因为模型在生成每个 token 时，已经通过权重进行了一部分“内部计算”。

可以把显式思考理解为写在纸上的草稿，把模型权重中的计算理解为专家脑中已经内化的模式。新手需要列很多步骤，因为许多关系还没有内化；专家能跳过一部分中间步骤，因为长期训练已经把结构压进直觉里。更大的预训练模型在某些任务上类似这种专家：它可能用更少显式 token 达到相同或更高质量。

这并不意味着大模型天然更便宜。每个 token 的计算成本可能更高；但 Yann 提到，大模型在推理时有更好的并行优化空间，整体系统效率未必下降。真实效率取决于模型规模、硬件并行、批处理、路由、缓存和服务系统。上一章讨论的测试时计算曲线，在这里多了一层基础模型因素：如果更大的模型把许多推理压进权重，显式 k 可能下降，延迟和质量的折中也会改变。

Section 4: 基础模型管线



教学重点：预训练继续扩展；数据约束推动合成、多模态和真实反馈组合，而世界模型必须接受现实校准。

图 7: 基础模型管线的教学视图：预训练吸收广泛先验，数据墙迫使训练配方转向高价值数据、合成数据、多模态数据和真实反馈；世界模型与模拟环境只有在持续接受现实校准时才有迁移价值。

视频来源区间：00:23:23–00:31:05；F2 抽样检查未发现可读幻灯片或标题卡，因此使用机制图。

不要把“更少显式思考”误读成“没有推理”

显式思考 token 减少，只说明模型把一部分计算压缩进权重和前向传播过程。它仍然在做复杂函数计算，只是未必把每个中间步骤写出来。对用户和产品来说，关键问题是输出是否可靠、是否可检查、是否在需要时愿意展开推理，而非表面上生成了多少草稿。

4.2 数据墙的真实含义：互联网数据、合成数据与高价值数据

Yann 承认，自己两年前也以为预训练可能会碰到墙；但从公开可观察的模型能力和成本信号看，行业似乎仍然能通过更大的模型获得很强表现。这里的“数据墙”需要精确定义，否则容易被误解成一句“互联网没数据了”的口号。

他在这里举了一个外部观察方式：如果不能谈内部训练细节，可以从公开价格或 per-token cost 粗略推断模型规模变化。他以 Anthropic 模型为例，说成本信号看起来像是更大的模型，而性能也确实很强。这个例子不应被写成确定的内部事实，它的价值在于展示一种带边界的外部推理：从成本、延迟和能力信号判断预训练扩展是否仍在产生收益。

数据墙

数据墙指高质量、可训练、可合法使用、能继续提供边际增益的数据增长速度，跟不上模型规模和训练需求增长的约束。它不只是数据总字节数不足；更关键的是高信噪比、覆盖稀缺任务、带有真实反馈、能提升模型短板的数据变得越来越难获得。

互联网文本仍然巨大，但模型越大，越需要更多样、更干净、更高价值的数据。低质量重复文

本、模板化网页、噪声论坛、SEO 内容，继续堆上去未必带来等比例收益。真正稀缺的是能教会模型处理复杂世界的的数据：专家推理过程、真实软件维护轨迹、高质量论文与实验记录、复杂工具使用日志、跨模态对齐样本、长期任务的成功和失败案例。

合成数据在数据受限时有价值。它可以由模型、程序、模拟器或规则系统生成，用来扩充稀缺任务、制造边界案例、提高覆盖率。数学证明步骤、代码修复样例、工具调用轨迹、对抗性问题，都可以通过合成流程放大。但合成数据有一个硬限制：如果生成过程只重复已有模型的偏差，训练会把偏差放大。合成数据像温室育苗，能快速扩繁；土壤若单一，苗长得整齐，却可能经不起真实气候。

高价值数据则更像少量但关键的矿石。它们未必规模大，却能改变模型在重要任务上的行为。真实专家标注、带验收结果的工作流、可靠的失败案例分析、跨工具长期任务轨迹，都属于这一类。对于前沿模型，数据价值常常来自“能不能提供新的约束”，而非“能不能再增加一批相似文本”。

Section 4: 数据来源与约束

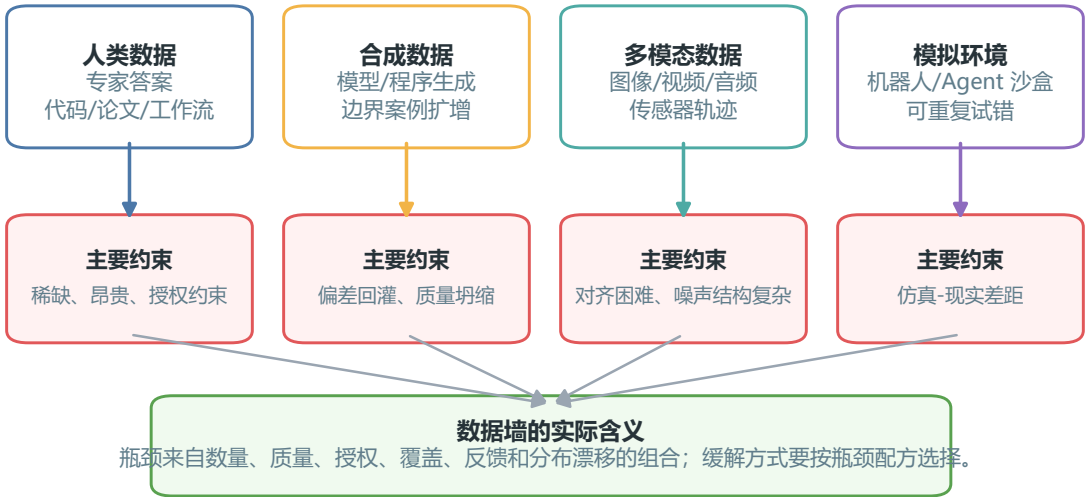


图 8: 数据来源与约束图：人类数据、合成数据、多模态数据和模拟环境分别缓解不同瓶颈，也分别带来授权、偏差、对齐和仿真-现实差距等约束。

视频来源区间：00:27:03–00:31:05；图中抽象了 Yann Dubois 对 *synthetic data*、*multimodal data*、*embodied AI* 和 *world model* 的讨论。

数据墙不是单一墙面

数据墙更像一组瓶颈：数量瓶颈、质量瓶颈、授权瓶颈、覆盖瓶颈、反馈瓶颈和分布漂移瓶颈。一个公司可能通过更大模型、合成数据、多模态数据、专家数据或更好的训练配方缓解某些瓶颈；这不等于所有瓶颈都消失。

4.3 多模态与具身智能：常识从哪里来

在 00:27:03–00:29:34，主持人问下一阶段数据前沿是否是多模态数据或合成数据。Yann 的回答带有明显保留：合成数据在数据受限场景可能有效，多模态数据也很有意思；他自己早年做过多模态表征学习，并长期认为多模态会帮助推理。但他也指出一个反例式观察：一些并不以多模态见长的模型依然非常聪明。这说明多模态对强文本模型并非简单的必要条件。

多模态数据的直觉价值在于，它把语言和世界其他投影连接起来。图像提供空间结构，视频提供时间和动作，音频提供节奏和语境，传感器数据提供物理状态变化。只读“球掉下去”这句话，模型可以学到语言规律；看见球从桌边滚落、加速下坠、撞击地面，模型获得的是另一种关于因果和物理连续性的证据。

具身智能进一步要求系统和世界互动。机器人、模拟代理、自动驾驶环境、游戏环境，都让模型或代理通过行动获得反馈。Yann 的表述很谨慎：他相信与真实世界互动会改善模型常识和对世界交互方式的理解，但整个学术界和 AI 社区距离这一目标仍然较远。这里的重点在于“交互”二字。静态数据教模型看见过去发生了什么；交互环境让模型学会自己的行动会改变什么。

常识这个词听起来朴素，实际很难。人知道杯子倒置后不能继续盛水，知道玻璃杯从高处摔下会碎，知道门把手通常需要旋转或下压，知道湿地面会滑。这些知识可以从文本中统计学习，也可以从视频和身体经验中获得。文本路线已经证明了惊人的上限；多模态和具身路线的价值，在于补上那些语言描述稀疏、物理反馈密集的部分。

读者可能忽略的一点：多模态不是“给模型加眼睛”这么简单

多模态训练的难点在于对齐。图像、视频、音频、文本和动作轨迹具有不同采样频率、噪声结构和语义粒度。真正有用的多模态数据要让模型学会跨表示预测同一个世界状态，而不只是把图片说明文字背下来。

因此，多模态与具身智能的贡献应当放在基础管线里理解。预训练负责吸收广泛先验，合成数据负责扩展受限分布，高价值数据负责纠正关键行为，多模态和交互数据负责补充世界结构。它们互相替代的空间有限，组合后的价值更大。

4.4 世界模型与模拟环境：什么时候该停止优化代理目标

在 00:29:34–00:31:05，主持人把话题引到世界模型。Yann 先确认语义：如果世界模型指复制或模拟一个环境，让代理在其中工作，那么问题在于模拟很难完全真实。模拟环境有价值，但它总会和真实世界存在差距，最终仍需要某种真实世界训练或校准，让模型意识到模拟和现实之间的错配。

世界模型

世界模型指系统对环境状态、动态规律和行动后果的内部或显式表示。它可以是神经网络中的隐式预测能力，也可以是可运行的模拟器。它的核心作用是让代理在行动前预估“如果我这样做，世界接下来会怎样”。

世界模型的吸引力很强，因为它承诺降低真实试错成本。机器人不必每次都摔坏真实硬件，自动驾驶不必在真实道路上测试所有危险场景，软件代理可以在沙盒里尝试修改。模拟环境像飞行

员训练舱：早期训练非常有用，可以安全、便宜、可重复地练习大量场景。

但训练舱永远带有近似。物理摩擦、传感器噪声、人类行为、长尾故障、硬件老化、真实组织流程，都会让模拟偏离现实。Yann 提醒，AI 领域常常会把某个模拟或代理目标优化到超过有用范围：一开始收益明显，继续做下去却开始服务于模拟本身。模型在模拟里得分越来越高，迁移到真实世界时却未必更好。

这里可以给出一个实用的停止条件框架。第一，看真实任务迁移收益是否还在增长。如果模拟分数上升，真实验证集或真实试点表现停滞，继续优化的价值就要被质疑。第二，看失败是否集中在模拟-现实错配处。如果错误来自真实环境中没有被模拟的变量，下一步应当补真实数据或改模拟器。第三，看代理是否学会利用模拟漏洞。只要策略开始钻规则空子，目标函数就需要重写或降权。第四，看边际成本。若一次模拟优化迭代消耗巨大，却只提高模拟指标，训练应转向更有信息量的真实反馈。

模拟环境的停止条件

当模拟指标继续上升、真实迁移收益不再改善，或代理开始利用模拟漏洞时，应停止单纯优化该模拟目标。下一步通常应转向真实世界校准、增加错配数据、修改环境假设，或把模拟目标降为辅助信号。

这个原则也反过来约束合成数据和世界模型研究。合成环境的价值来自可控、可扩展、可重复；风险来自闭环自我强化。好的基础模型管线需要在“便宜的大量近似反馈”和“昂贵的真实反馈”之间做配比。只靠真实世界试错，成本太高；只靠模拟，目标容易漂移。

4.5 本章小结

本章说明了基础模型管线仍然是前沿进展的底座。预训练并未在视频语境中被判定为停止扩展；相反，Yann 强调模型仍在变好，并指出更大的模型可能把一部分显式思考压进权重，从而减少思考 token。这个机制需要和推理服务效率一起看：大模型单 token 更贵，但并行和系统优化可能改变整体延迟-质量折中。

数据墙的含义也需要拆开看。它指高质量、可用、能提供边际增益的数据越来越稀缺，涉及数量、质量、授权、覆盖和反馈等多重瓶颈。合成数据可以缓解数据受限分布，高价值数据能提供稀缺约束，多模态和具身智能则补充语言之外的世界结构。世界模型和模拟环境有助于低成本试错，但优化必须在真实迁移收益下降或模拟漏洞出现时及时停止。基础模型管线的关键判断是：哪些信息应从规模获得，哪些应从高价值数据获得，哪些必须从真实世界反馈获得。

5 后训练的核心机制：SFT、行为克隆与强化学习

5.1 中训练：把高质量数据的权重调高

Yann Dubois 在这一段先把训练管线重新分层：预训练负责让模型从大规模互联网语料中吸收广泛知识，中训练位于预训练和后训练之间，目标是把更接近最终模型需求的高质量数据加权得更重。这里的关键词是“代表性”和“过采样”。互联网上有百科、代码、论文，也有广告、重复页面、低信息密度论坛帖；全部数据都能提供语言模式，却不都同等有助于训练一个有用的前沿模型。

中训练可以理解为给模型的阅读清单重新排序。预训练像把整座图书馆扫进模型参数里；中训练像告诉模型，接下来请多读那些更可靠、更专业、更接近目标任务的书架。它不必改变“模型从文本预测中学习”的基本形式，却会改变训练分布，让模型在后续阶段拥有更好的起点。

中训练的定位

中训练指位于预训练和后训练之间的训练阶段。它通常提高高质量数据、目标领域数据或更具代表性数据的训练权重，使基础模型在进入 SFT 和 RL 之前，已经更接近最终要服务的任务分布。它像在大规模通识教育之后安排一段专业强化课。

这个阶段容易被低估，因为它不像 RL 那样有“奖励最大化”的戏剧性，也不像预训练那样有参数规模和数据规模的直观冲击。实际工程里，数据分布的微小移动会改变后面所有阶段的效率。若中训练已经让模型更常见到高质量推理、代码、工具使用和专业写作，后训练就不需要从很远的地方把行为硬拉回来。

Section 5: 训练管线从知识到行为

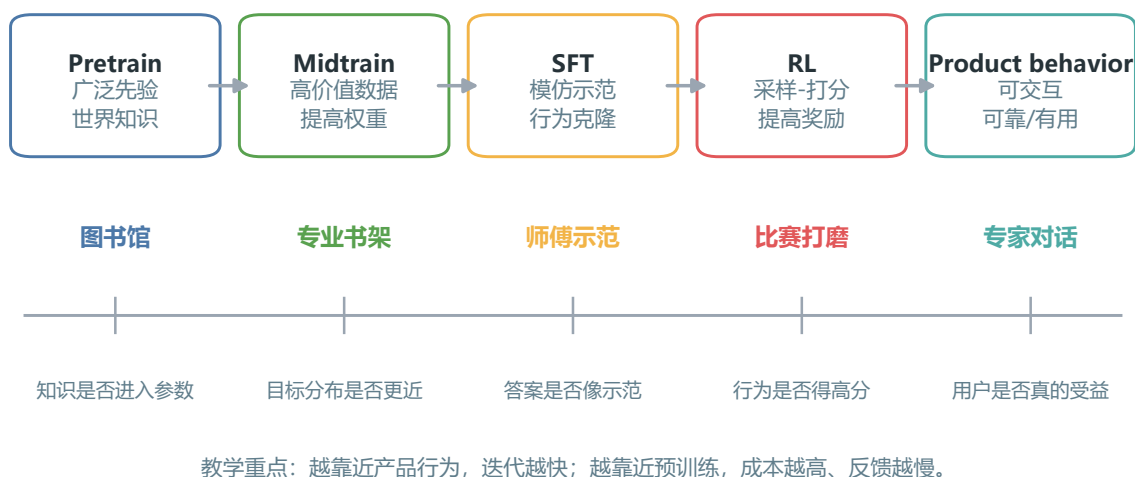


图 9: 从知识到行为的训练管线：pretrain 提供广泛知识，midtrain 提高目标数据权重，SFT 复制示范行为，RL 通过奖励筛选更好轨迹，最终落到可交互的产品行为。

视频来源区间：00:31:05–00:37:21；图对应中训练、后训练、SFT、行为克隆和 RL 的分层解释。

5.2 后训练：把会很多知识的模型变成可交互专家

Yann 给后训练的解释非常适合教学：预训练后的模型像一座图书馆，里面有大量信息；后训练要把这座图书馆变成一个能回答问题、理解意图、配合用户工作的专家。图书馆有书，专家能对话、能追问、能判断你真正需要哪一页。

后训练因此不只是“让模型更礼貌”。它承担的是行为塑形：模型如何遵循指令，如何承认不确定，如何调用工具，如何在可验证任务中寻找正确答案，如何在开放任务中更接近用户效用。

SFT、偏好优化、RL、推理模型训练都可以落在这个大伞下；本节重点放在源材料中反复强调的两条主线：SFT/行为克隆，以及基于奖励函数的强化学习。

后训练的核心定义

后训练指预训练之后的一组训练过程，其目标是把“知道很多”的基础模型塑造成“对用户有用、可交互、可控制”的模型。它把关注点从知识是否曾经进入参数，转向模型能否在具体输入下产生合适的行为。

这个定义也划清了一个边界：后训练并不等于单一算法。SFT 是后训练，RL 也是后训练，围绕工具调用、指令遵循、拒答边界、推理时间控制的训练同样属于后训练。后续讨论 RL 时，读者要记住它只是后训练管线的一部分；若把全部进步都归功于 RL，就会错过中训练、数据筛选、评测设计和产品约束的共同作用。

5.3 SFT 与行为克隆：从人类答案中复制行为

SFT，即监督微调，使用“输入 prompt + 目标答案”的示范数据训练模型。若人类标注者、专家或教师模型给出了期望答案，训练就让模型在类似输入下提高生成该答案的概率。Yann 把这称为行为克隆：给模型看人类怎么做，然后让模型模仿。

可以把 SFT 想成师傅带徒弟。师傅先把标准动作做给徒弟看：面对一个问题怎样组织答案，怎样写代码，怎样解释概念，怎样拒绝不该回答的请求。徒弟通过大量示范形成肌肉记忆。这个方法稳定、直接、工程上容易启动，所以早期许多后训练模型主要依赖 SFT。

SFT 与行为克隆

SFT 是 LLM 领域最常见的行为克隆形式：训练数据给出理想输入输出对，模型学习在相似输入下复制这些示范行为。它的优势是高效、稳定、可控；限制是上限强烈受示范数据质量约束。

行为克隆的硬边界也在这里：模型通常很难系统性超过示范者。若示范答案只是“看起来像好答案”，模型会学会这种外观；若示范者无法完成高难度推理，模型也缺少从示范中超越的信号。更麻烦的是，SFT 会把答案风格、格式习惯和隐含错误一起复制。人类答案里经常有引用、结论、分步骤推理，模型可能学到“这种语境下应该给出一个像样答案”，却没有学到每个事实都要可验证。

这一点解释了为什么 SFT 是好起点，却很少是终点。它先把模型推到一个可用区域，让模型不至于在后续 RL 中随机乱撞；随后 RL 才有机会通过奖励信号继续筛选更好的行为。

5.4 强化学习：从奖励函数中超过示范答案

强化学习的核心转换是：训练者未必知道完美答案长什么样，但可以判断某个答案是否更好。Yann 用很朴素的语言说，RL 会让模型多次采样答案，然后告诉它“这个正确，那个不正确，多做正确的那类”。这看似简单，背后改变了训练信号的性质：SFT 追随示范，RL 优化奖励函数。

奖励函数 $R(x, y)$ 是把输出质量变成分数的规则。数学题可以用最终答案是否正确给二元奖励；代码可以运行测试；开放式问答可能使用成对比较、专家打分或模型裁判。奖励越可靠，RL

越像有清楚靶心的射击训练；奖励越模糊，RL 越像在雾里调罗盘，仍有方向，但噪声大得多。

一个常用的高层 RL 目标可以写成下面的形式。它表达的是：模型希望提高高奖励答案的概率，同时不要偏离参考模型太远。

$$J(\theta) = \mathbb{E}_{x \sim D, y \sim \pi_\theta(\cdot | x)} [R(x, y)] - \beta \text{KL}(\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)).$$

符号解释：

- $J(\theta)$ ：训练目标，表示参数为 θ 的模型策略在奖励和约束之间取得的综合分数。
- $x \sim D$ ：从训练任务分布 D 中采样输入。 x 可以是数学题、代码任务、用户请求或 Agent 任务起点。
- $y \sim \pi_\theta(\cdot | x)$ ：当前可训练模型在输入 x 下采样出的输出。这里的 π_θ 是模型策略，也就是“给定输入后生成各种答案的概率分布”。
- $R(x, y)$ ：奖励函数，对输入 x 与输出 y 的组合打分。分数越高，训练越倾向于提高这类输出的概率。
- π_{ref} ：参考策略，通常是 SFT 后或上一阶段得到的稳定模型。它像一条安全绳，防止 RL 为了追逐奖励把模型行为拉得太偏。
- $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ ：KL 散度，用来衡量当前模型分布和参考模型分布的差异。差异越大，惩罚越大。
- β ：KL 惩罚强度。较大的 β 会让模型更贴近参考模型，较小的 β 允许模型为了奖励做更大行为移动。

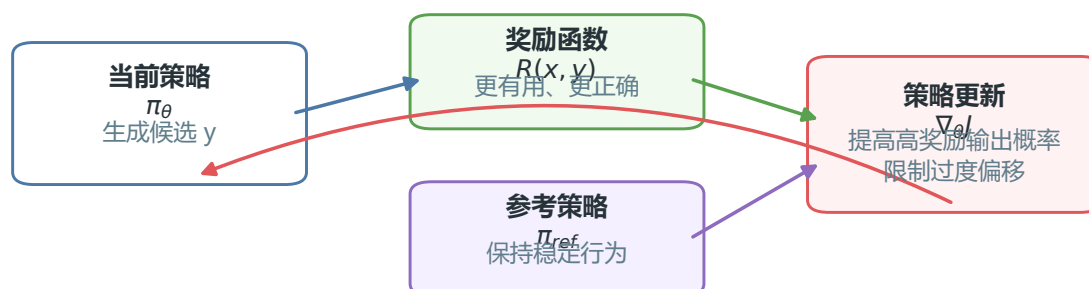
KL 项还可以展开为更直观的形式：

$$\text{KL}(\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)) = \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)} \right].$$

符号解释：

- $\log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}$ ：当前模型相对参考模型更偏好某个输出 y 的程度。
- 外层期望 $\mathbb{E}_{y \sim \pi_\theta}$ ：按照当前模型实际会采样到的答案来平均这种偏离。
- 这个 KL 项的直觉作用是限制“奖励黑客”。如果奖励函数有漏洞，KL 惩罚能降低模型突然学出怪异捷径的概率。

Section 5: KL 约束下的 RL 目标



$$J(\theta) = \mathbb{E}[R(x, y)] - \beta \text{KL}(\pi_\theta \| \pi_{ref})$$

奖励项像油门，KL 项像护栏； β 越大，模型越贴近参考行为。

图 10: KL 约束下的 RL 目标：奖励项推动模型提高高质量输出概率，KL 项限制策略相对参考模型的偏移，二者共同决定一次策略更新的方向。

视频来源区间：00:37:21-00:38:53；图服务于正文中的 $J(\theta) = \mathbb{E}[R(x, y)] - \beta \text{KL}(\pi_\theta \| \pi_{ref})$ 解释。

奖励函数不是愿望清单

奖励函数必须能被执行、被评估、被规模化调用。人类说“答案要有帮助、真实、简洁”只是价值描述；训练系统还要把这些要求落成打分器、比较器、测试、专家评审或模型裁判。奖励写得含糊，RL 会优化含糊之处；奖励覆盖不到的行为，模型可能继续犯错。

这也解释了为什么可验证奖励曾经特别适合数学和代码。答案能自动检查，奖励信号干净，模型容易知道哪类采样值得增加概率。开放式真实任务则更棘手，因为“更好”往往依赖上下文、用户偏好、风险边界和专业判断。

5.5 RL 是否创造新能力：预训练潜能与后训练显化

主持人问了一个很关键的问题：RL 到底创造新能力，还是让模型更会使用既有能力？Yann 的回答非常谨慎：科学上很难干净地区分。预训练在全互联网规模上学习，许多潜在能力可能早已以弱形式存在于模型参数中；后训练把这些能力调出来、稳定下来、组织成可用行为。

这个边界可以用“矿石”和“刀具”作类比。预训练像把矿石、合金和材料性质放进仓库；SFT 像教模型按人类示范打磨成基础工具；RL 像反复用真实切割任务检验刀刃，把更锋利、更可靠的形状保留下来。最后出现的刀具在使用体验上确实像新东西，但很难说每一分锋利都来自最后一道工序。

能力问题的精确边界

若从参数来源看，许多能力可能在预训练中已有潜在表征；若从可观察行为看，RL 可以让模型表现出此前很少稳定出现的能力，例如检查答案、延长思考、修正错误、选择更高奖励的推理路径。更严谨的说法是：RL 常常显化、放大并稳定预训练中的潜能，也可能通过大规模采样和奖励优化形成新的行为组合。

源材料里有一个规模信号很重要：Yann 提到早期 Alpaca 级别的 SFT 只用了约 5 万示例，而后来的开源推理模型和 DeepSeek 相关模型已经把 RL 阶段扩展到接近百万级数据点。规模改变了结果。小规模 RL 容易显得像微调边角；大规模 RL 加上强基础模型和好奖励，会让模型稳定学会“想更久、检查自己、改进答案”这类推理行为。

读者需要警惕两个极端说法。第一种极端把 RL 神化，好像奖励函数能从空白模型里凭空造出专家。源材料明确说，从随机行为开始做 RL 效率极低，因为模型必须先“撞上”好答案。第二种极端把 RL 轻描淡写成最后一点装饰。过去两年开源社区的经验显示，当基础模型有足够好先验，RL 的规模化会明显改变模型可见能力。

5.6 本章小结

本章把后训练拆成了几个清晰层次。中训练通过提高高质量数据权重，让基础模型更接近最终任务分布；后训练把“会很多知识”的模型塑造成可交互、可控制、对用户有用的专家。SFT 是行为克隆：它从人类或教师模型示范中复制答案模式，稳定高效，但上限受示范质量约束。

RL 的关键变化在于从复制答案转向优化奖励。目标 $J(\theta)$ 同时包含奖励项和 KL 约束： π_θ 负责生成可训练行为， π_{ref} 提供参考边界， $R(x, y)$ 给出质量信号， β 控制偏离参考模型的力度，KL 散度约束模型别为了奖励跑偏。至于 RL 是否创造新能力，更精确的答案是：它通常把预训练中潜在的能力显化、放大、稳定，并在大规模采样和奖励优化下形成新的行为组合。

6 扩展 RL 的工程现实：采样、归因、GRPO 与工艺科学化

6.1 为什么 RL 曾经显得难以扩展

Yann Dubois 对 RL 的历史判断很直白：直到两年前，许多没有长期做 RL 的研究者都觉得它太难调、太不稳定、太不值得。ChatGPT 发布后，外界看到 OpenAI 使用了强化学习，很多人的第一反应仍然是：能不能只用 SFT 复现同样效果？Stanford Alpaca 一类项目正是在这种背景下出现的，思路是用行为克隆尽量逼近对话模型的可用性。

这种怀疑有现实基础。传统 RL 经常面临稀疏奖励、探索成本高、训练不稳定、环境难以复现、超参数敏感等问题。若模型还没有足够好的先验，它在巨大的输出空间里采样，命中高质量答案的概率很低。就像让一个完全没学过棋的人靠输赢反馈自学围棋：每盘都能知道输赢，但很难知道第 37 手到底错在哪里。

好先验为何改变 RL 的命运

先验指模型在 RL 开始之前已经拥有的知识、技能和行为倾向。强 LLM 已经懂语言、代码、常识、任务格式和大量世界知识；RL 不需要从零探索“什么是像样答案”，它只需要在已有

能力附近筛选更高奖励的轨迹。先验越好，采样越容易撞上可奖励行为，RL 的效率越高。

Yann 提到一个跨领域信号：机器人也正在进入类似阶段。过去机器人 RL 同样显得不稳定，因为系统要在物理或仿真空间里从低级动作开始探索；当视觉语言模型和世界知识模型提供更强先验后，机器人学习也更容易站到“已经知道大概该怎么做”的位置上。LLM 和机器人之间的共性是：RL 最怕从空白开始，最喜欢在有用行为附近细调搜索。

6.2 好先验让 RL 变得可用：从 LLM 到机器人

好先验的价值可以用搜索空间来理解。若模型完全不知道答案分布，RL 就像在巨大城市里随机找一扇门；若 SFT、中训练和预训练已经把模型带到正确街区，RL 只需要比较同一条街上的几扇门。后者仍然要采样、打分、更新，但每次采样携带的信息密度高得多。

这也是为什么第 5 章强调“先行为克隆，再 RL”。源材料中 Yann 说得很清楚：如果一开始就做 RL，会非常低效，因为模型必须先偶然采到正确答案。SFT 把模型推进人类可接受行为，RL 再从这些候选行为中寻找更高奖励的变体。两者的关系像先学会基本笔画，再通过比赛和评分打磨书法风格。

从 LLM 到机器人，工程含义是一致的：RL 不是单独的魔法开关，它依赖上游表示、数据、环境、奖励和采样基础设施。一个有视觉、语言和常识先验的机器人，在厨房任务里不必从“手臂可以移动”开始学起；一个经过大规模预训练和 SFT 的 LLM，在代码任务里不必从“代码由字符组成”开始探索。

扩展 RL 的第一原则

RL 能否扩展，首先取决于模型是否已经处在“可能采到好行为”的区域。预训练、中训练和 SFT 提供先验，奖励函数提供方向，采样系统提供规模。缺一项，RL 都会显得昂贵、脆弱或投机。

6.3 采样成本与长 rollout 的信用分配问题

扩展 RL 的第一类困难是基础设施成本。RL 要从当前模型里采样许多答案或行动轨迹，再对它们评分。对 LLM 来说，每个候选答案都要消耗推理算力；对 Agent 来说，一个候选轨迹还可能包含多轮工具调用、浏览器操作、代码运行、文件修改和环境等待。采样越多，奖励估计越稳定，成本也越高。

第二类困难更偏机器学习：长 rollout 的信用分配。rollout 指模型从起点出发完成的一整段行为轨迹。短任务里，模型给出一个答案，奖励很快回来；长 Agent 任务里，模型可能先读文件、再写代码、再运行测试、再修复、再汇报。最终成功或失败发生在最后，训练系统却要判断前面哪一步真正造成了结果。

长 rollout 的信用分配困难

长 rollout 只在终点给出成功或失败信号时，每个 token、每次工具调用、每个中间决策得到的信息都很稀薄。模型可能因为早期误读需求失败，也可能因为最后一步格式错误失败；如果奖励只说“失败”，训练就很难知道该降低哪类行为的概率。Agent 越长、工具越多、环

境越复杂，信用分配越像在一段长流水账里追查真正的事故源头。

这件事可以用策略梯度的直觉式写法表示。训练大致会提高“带来正优势”的输出概率，降低“带来负优势”的输出概率：

$$\nabla_{\theta} J \approx \mathbb{E}[A(x, y) \nabla_{\theta} \log \pi_{\theta}(y | x)].$$

符号解释：

- $\nabla_{\theta} J$ ：训练目标对模型参数的更新方向。
- $A(x, y)$ ：优势值，表示输出 y 相对某个基线有多好。正优势鼓励模型多生成类似输出，负优势抑制类似输出。
- $\nabla_{\theta} \log \pi_{\theta}(y | x)$ ：提高当前输出概率时参数需要移动的方向。
- 这个式子只表达高层直觉：RL 更新依赖“哪个输出比基线更好”的判断。长 rollout 的难点在于 $A(x, y)$ 很难被准确分配到具体中间步骤。

如果把长 Agent 任务比作一场接力赛，最终名次只告诉团队赢了或输了，却没有告诉每一棒的速度、交接失误和路线选择。工程上需要更密的日志、更分层的奖励、更好的中间检查点，才能把“最后失败”拆成可训练信号。

Section 6: 长 rollout 的信用分配

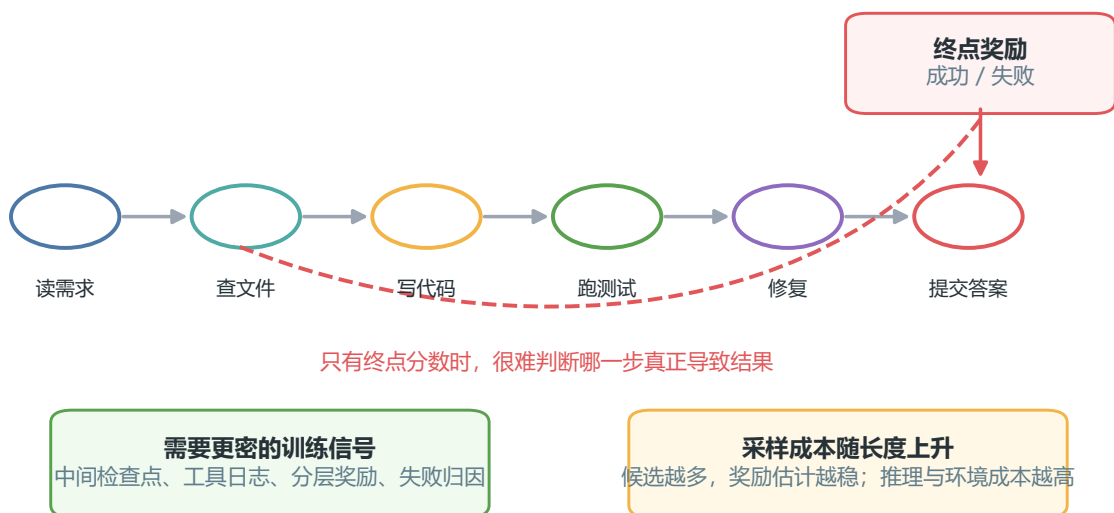


图 11: 长 rollout 的信用分配问题：Agent 任务可能包含读需求、查文件、写代码、跑测试和修复等多步行为；若奖励只在终点出现，训练系统很难判断哪一步真正导致成功或失败。

视频来源区间：00:38:53-00:43:09；图对应采样成本、长轨迹和 attribution 难题。

6.4 GRPO、PPO、DPO：可扩展方法为何胜出

主持人提到 GRPO、PPO、DPO 这一串缩写时，Yann 没有展开内部细节，也明确不能谈 OpenAI 内部做法。他给出的开放社区观察是：GRPO 看起来效果很好，而机器学习里经常出现一种规律，可规模化的简单方法最终占优。

高层看，PPO、DPO、GRPO 的区别可以这样理解。PPO 是较经典的策略优化方法，通常通过限制每次策略更新幅度来保持训练稳定；它功能强，但工程部件较多。DPO 直接利用偏好数据优化模型，让“被偏好的答案”相对“不被偏好的答案”概率更高，常用于偏好对齐场景，形式上避开了部分在线 RL 复杂性。GRPO 则把同一输入下的一组采样答案放在一起比较，用组内相对表现构造优势信号。

GRPO 的直觉公式可以写成组内相对优势：

$$A_i = \frac{r_i - \bar{r}_G}{\sigma_G + \epsilon}, \quad \bar{r}_G = \frac{1}{G} \sum_{j=1}^G r_j, \quad \sigma_G = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \bar{r}_G)^2}.$$

符号解释：

- G ：同一个输入下采样出的答案数量，也就是组大小。
- r_i ：第 i 个采样答案的奖励分数。
- $\bar{r}_G$ ：该组答案的平均奖励，作为组内基线。
- σ_G ：该组奖励的标准差，用来把不同组的奖励尺度归一化。
- ϵ ：很小的正数，防止 σ_G 接近零时除法不稳定。
- A_i ：第 i 个答案的组内相对优势。它衡量这个答案比问题其他采样好多少，而不是只看绝对分数。

这个公式的教学价值在于，它把“多采样、比较、强化更好答案”的直觉写清楚了。若同一道题采样出 8 个答案，奖励最高的几个会得到正优势，明显差的会得到负优势。模型更新不必先知道“完美答案”的完整形状，只需要知道同一组候选里谁更好。

Section 6: GRPO 组内相对优势草图

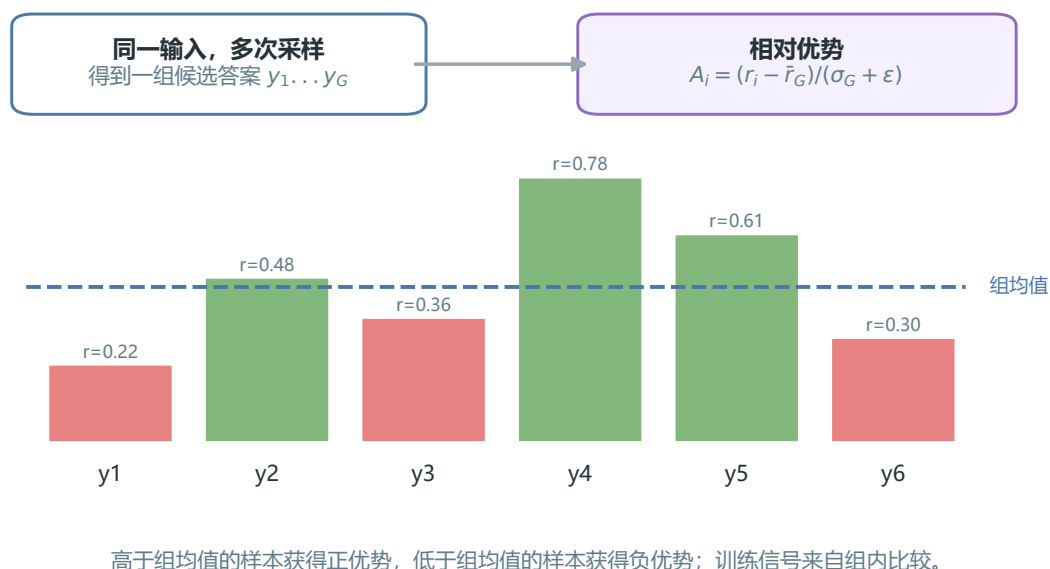


图 12: GRPO 的组内相对优势草图：同一输入下采样一组候选答案，用组均值和组内方差把奖励转为相对优势，从而强化高于组内基线的样本。

视频来源区间：00:43:09–00:48:21；图只呈现高层直觉，细节以正文符号解释为准。

不要把缩写当作护身符

PPO、DPO、GRPO 的差异重要，但源材料的重点不在算法名字本身。真正的工程问题是：方法能否稳定训练，能否承受大规模采样，能否用有限基础设施跑出足够多候选，能否把奖励噪声控制在可接受范围。缩写会变，扩展约束长期存在。

为什么简单方法常常胜出？因为在大规模机器学习里，算法的“理论精巧”必须穿过基础设施、吞吐、调参、人力和失败恢复。一个稍粗糙但能稳定扩大 10 倍采样的方法，常常会超过一个更精致但难以扩展的方法。Yann 的说法提醒读者：前沿训练不是纸面最优算法比赛，它是算法与系统共同承压的工程过程。

6.5 从手艺到科学：复杂系统中的研究节奏

视频后半段转向一个更元的问题：AI 模型到底是被“建造”出来的，还是被“培育”出来的？Yann 的回答是，机器学习常常从手艺开始，再逐渐走向科学。早期阶段，人们尝试许多做法，凭经验和直觉建立心智模型；某些做法跑通后，研究者再系统化地分析、消融、测量和改进。

这里的“手艺”不是贬义。复杂系统里，很多变量互相耦合：数据分布、模型规模、优化器、奖励、推理服务、评测、产品路由都会影响结果。刚进入一个新区域时，严格理论往往还没有足够对象可解释，研究者只能先靠实验手感摸到可行路径。等路径稳定后，科学化才有抓手：哪些因素真的有效，哪些只是偶然，哪些可以被规模化复现。

工艺科学化的节奏

前沿模型训练常见节奏是：先用手艺找到可行现象，再用科学方法把现象压实。手艺负责打开局面，科学负责降低偶然性、提高可复现性、把经验变成可扩展流程。

这也解释了为什么越靠近最终模型阶段，迭代越快。后训练和产品侧改动常常可以按天迭代；中训练和预训练越往上游走，周期越长、成本越高、反馈越慢。团队通常会先在后训练阶段快速试错，确认信号后再把改变向更深的训练栈传导。开源社区里后训练模型远多于全新预训练基座，也符合这个节奏：最后阶段成本较低，更多人能实验。

源材料末尾还给第 7 章埋下伏笔：模型能力的凹凸不平，往往和“训练的是哪类能力”有关。若优化数学竞赛，模型可能迁移到代码竞赛，因为两者共享形式推理、搜索和验证能力；若模型在某件事上很差，问题可能跨语言、跨领域出现，因为缺的是更底层的横向能力。RL 扩展因此不能只按行业标签理解，还要按能力轴理解。

6.6 本章小结

本章说明了 RL 扩展的工程现实。RL 曾经显得难以扩展，因为它采样昂贵、奖励稀疏、训练敏感，还常常需要从低概率好行为里探索信号。强预训练模型、中训练和 SFT 提供好先验后，RL 才更容易在有用行为附近做搜索；同样逻辑也正在影响机器人等具身系统。

真正的瓶颈随后转向系统成本和信用分配。大规模采样需要推理基础设施，长 rollout 让成功或失败信号难以归因到具体 token、工具调用或中间决策。GRPO 的组内相对优势 $A_i = (r_i - \bar{r}_G)/(\sigma_G + \epsilon)$ 提供了一种直观思路：问题多采样，组内比较，把更好的候选推高。PPO、DPO、GRPO 的细节各异，但可扩展、稳定、简单的方法在大规模系统里更容易胜出。前沿训练的节奏也由此清晰：先靠手艺找到能工作的现象，再靠科学化测量与工程化扩展把它变成可靠流程。

7 跨领域泛化与幻觉控制：从 GDPval 到负迁移

7.1 能力如何跨领域扩散：算法、数据与关注者数量

Yann Dubois 在这一段里把“泛化”拆得很实在：模型能力从数学、代码扩散到经济任务，既有底层算法的可复用，也有数据、奖励、专家关注度的重新分配。把它想成一套工业生产线会更直观：算法像机器，数据像原料，奖励函数像质检标准，真正盯着产品缺陷的人像工艺工程师。机器可以搬到新工厂，原料和质检标准却要重新组织；若没有懂该行业的人持续看输出，模型很难快速变好。

视频里提到 GDPval 时，语境很明确：它代表一种面向经济活动的评测方向，用来观察模型在不同经济领域任务上的表现。这里的重点不在某个单项分数，而在研究重心的迁移：从数学竞赛、编程竞赛这类“题目完全写在纸上”的任务，转向咨询、金融、法律、医疗等需要查资料、判断上下文、处理缺失信息的任务。

GDPval：视频语境中的经济任务评测

本章只按视频和本地 metadata 的描述来理解 GDPval：它是围绕“经济中不同领域的任务”评估模型表现的 benchmark family。它在这段谈话中的功能，是说明评测对象开始从窄而

清楚的竞赛题，扩展到真实经济活动中的开放任务。这里不额外外推它的完整题库、评分细节或官方指标。

Dubois 给出的第一条判断很硬：模型在哪些垂直领域进步快，往往取决于有多少真正关心该输出的人在用它。编程进步快，一个原因是训练模型的人自己天天写代码，能直接感到哪些行为有用、哪些行为糟糕。法律、医疗、宏观经济分析的问题更加麻烦，因为模型团队里的专家密度较低，反馈链条更长，错误成本也更高。

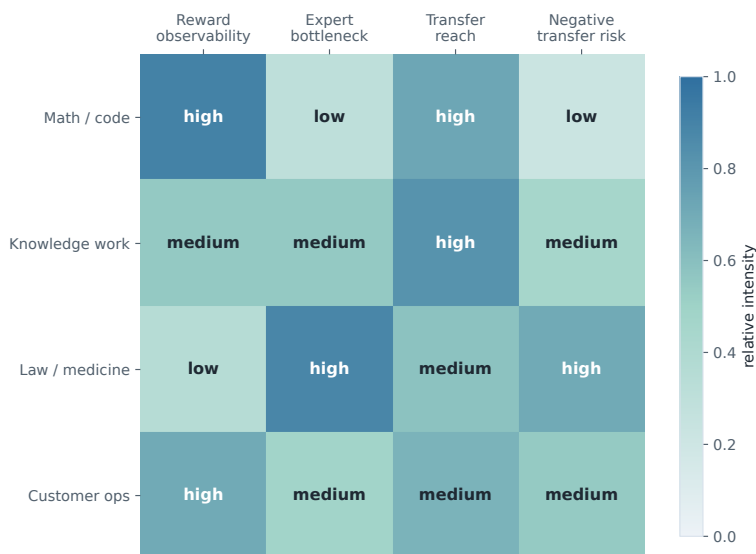
这形成一个近似的工程关系。下面的符号只服务于本节的教学类比，不进入全篇共享符号表；其中 C_{alg} 、 D_d 、 H_d 、 E_d 分别压缩算法、数据、专家和评测环境四类因素：

$$\Delta U_d \propto C_{\text{alg}} \cdot D_d \cdot H_d \cdot E_d$$

- ΔU_d ：领域 d 中模型效用的增量，例如法律、医疗、金融或代码领域的可用性提升。
- C_{alg} ：可迁移的算法能力，例如同一类 RL 方法能否搬到新任务上。
- D_d ：领域 d 的高质量数据与任务样本。
- H_d ：真正懂该领域、愿意盯模型输出的人类专家数量与投入强度。
- E_d ：可执行的评测与奖励环境，包含自动判分、专家打分、工具验证等机制。

这个式子不是视频里的精确定律，只是把 Dubois 的口头判断压缩成一个工程模型。它提醒读者：通用模型并不会凭空“均匀照亮”所有行业。行业越缺数据、缺专家、缺可验证奖励，进步越可能慢。

Generalization depends on what transfers and what can be scored



Rule of thumb: trainable domains combine observable feedback, reusable horizontal skills, and manageable expert review.

图 13: 跨领域泛化矩阵：该图把任务族放在奖励可观测性、专家瓶颈、迁移范围和负迁移风险四个维度上，帮助第 7 节解释为什么 GDPval、法律、医疗和客服工作流的训练难度主要来自反馈质量与专家稀缺。

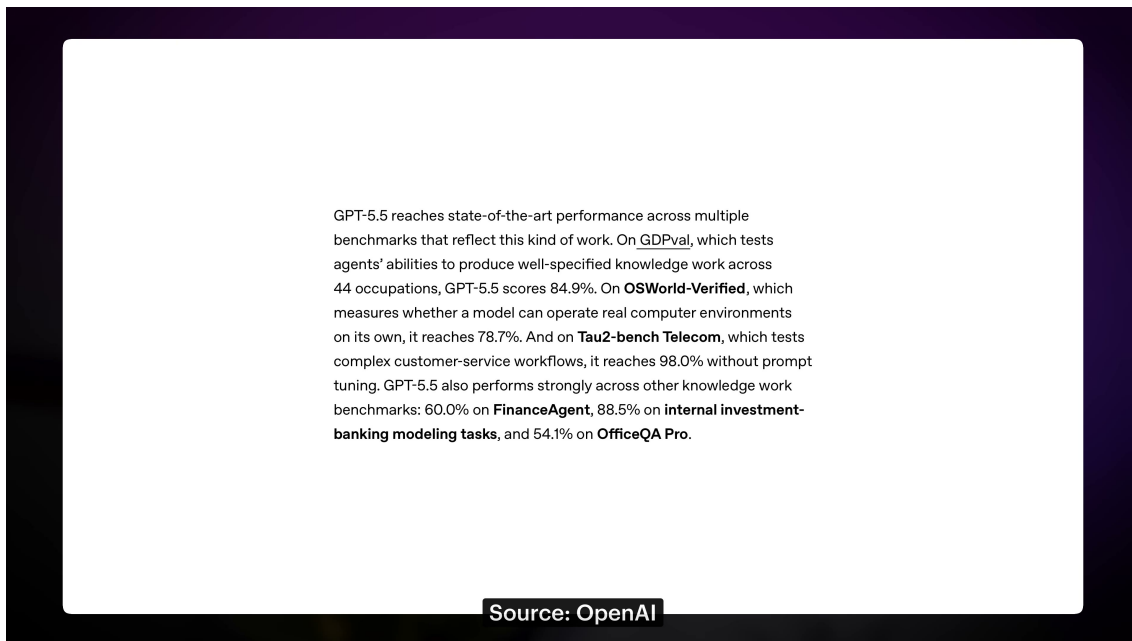


图 14: 视频中的经济任务基准评测画面: 画面列出 GDPval、OSWorld-Verified、Tau2-bench Telecom、FinanceAgent 和 OfficeQA Pro 等评测名, 适合连接第 7 节的经济任务评测讨论与第 9 节的垂直应用、最后一公里讨论。³

7.2 横向能力的泛化: 编辑、指令遵循与多语言迁移

Dubois 区分了两类泛化。第一类是算法泛化: 在领域 A 中发展出来的训练方法, 能否用于领域 B。他举了开源社区里 GRPO 被迁移到许多任务上的例子, 说明有些 RL 方法确实像通用工具箱, 换一个任务仍能工作。

第二类是模型能力泛化: 模型在某个数据集上训练之后, 能力是否会扩展到其他领域。这里应把关注点放在“能力轴”, 少被“领域名”牵着走。如果训练提升的是编辑能力、遵循格式能力、调试能力、校准能力、信息检索能力, 那么这种能力可能跨语言、跨任务、跨行业扩散。一个模型用很少的 C++ 专门训练也能写好 C++, 原因之一是预训练已经见过大量 C++, 后训练只需把潜在能力调出来。

更细一点看, 横向能力像人体的肌群。数学竞赛能力像某个专项动作, 隐含信息检索、问题澄清、工具使用、事实校准等能力像核心肌群。专项动作能拿高分, 不保证核心肌群足够强; 一旦任务从五行题面变成真实咨询项目, 模型就要先判断缺什么信息、去哪里找资料、哪些证据可信, 再开始推理。

泛化的正确粒度

在本章语境里, 泛化不应按“法律、医疗、代码”这种行业标签粗暴理解。更有解释力的粒度是能力轴: 能否识别缺失信息, 能否拒绝编造, 能否遵循显式要求, 能否理解隐式意图, 能否把工具调用嵌入推理流程。行业只是这些能力的组合场。

这也解释了一个常见误判: 人类里数学好的人常被认为“聪明”, 于是可以迁移到许多问题;

³视频帧来源时间区间: 01:11:33–01:12:09。

模型在数学竞赛上强，未必会自动会处理开放世界任务。竞赛题通常把全部条件放在 prompt 里，真实经济任务则要求模型走出题面，自己寻找证据和约束。这个差别很像封闭书考试与真实工作之间的差别：前者考解题，后者先考识别问题。

7.3 SFT 为什么会放大幻觉，RL 为什么能惩罚编造

幻觉在这里指模型自信地产生没有依据或错误的内容，典型表现是编造引用、编造事实、在不知道时仍给出确定答案。Dubois 的观察是：如果一个模型缺乏“知道自己不知道”的能力，这个问题通常会跨领域出现。它不会只在医学里诚实、在金融里胡编；校准能力本身也是横向能力。

John Schulman 的相关观点在视频里被用来解释 SFT 的风险。监督微调让模型模仿人类给出的标准答案。若标准答案引用了一篇论文，而模型内部并不知道这篇论文，那么训练信号会把“在这种语境下给出论文引用”强化成行为模式。站在模型分布的角度，它学到的可能是“回答时应该出现看似权威的引用”，并没有真正学到“只有确认来源存在时才引用”。

可以用一个简单的条件概率来描述这件事：

$$P_{\text{cite}}(y \mid x, \text{unknown}) \uparrow$$

- x ：用户问题或任务上下文。
- y ：模型生成的答案。
- unknown：模型没有足够内部知识或外部证据支持某个事实。
- P_{cite} ：答案中出现引用、来源、论文名等权威化表达的概率。
- $\uparrow$ ：经过模仿式训练后，该行为概率可能上升。

RL 的逻辑不同。模型先采样答案，再由奖励函数判断好坏。若模型不知道某篇论文，随机采样出一个真实、相关、格式正确且可验证的引用，概率极低；更常见的采样结果是编造或错引。只要奖励系统能识别这些错误，编造行为就会被扣分，策略会逐渐降低这类输出的概率。

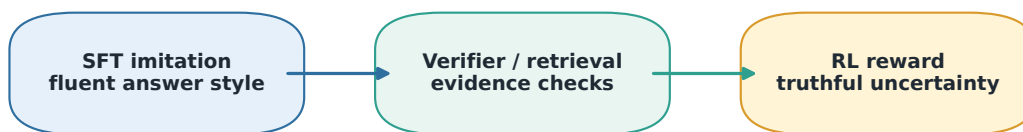
$$\Delta \log \pi_{\theta}(y \mid x) \propto R(x, y)$$

- $\pi_{\theta}(y \mid x)$ ：当前模型在输入 x 下生成答案 y 的概率。
- $\Delta \log \pi_{\theta}(y \mid x)$ ：训练后该答案概率的变化方向。
- $R(x, y)$ ：奖励函数对答案的评分；编造引用、无依据断言、错误事实应得到低分。
- $\propto$ ：表示概率变化方向与奖励信号相关，这里只表达直觉关系。

幻觉控制的前提

RL 能惩罚编造，前提是奖励系统看得见编造。若评测器无法核验事实、无法检查引用、无法区分合理不确定性与逃避回答，RL 也会学到新的投机方式。幻觉控制的核心瓶颈常常落在“可验证性”和“评测质量”上。

Hallucination control: shift reward toward supported answers



The control target is behavior under uncertainty: cite, verify, ask, or say "I do not know" when support is missing.

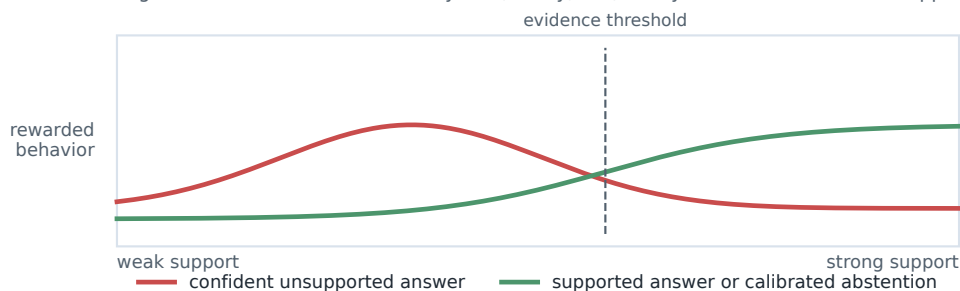


图 15: 幻觉控制机制图: SFT 容易强化流利回答风格, 检索或 verifier 提供证据检查, RL 奖励把行为推向有依据回答、校准拒答和承认不确定性。

7.4 负迁移: 显式指令遵循与隐式意图理解的冲突

当主持人追问“在一个领域变强会不会让其他领域变弱”时, Dubois 的回答分两层。垂直领域之间通常更多是资源分配问题: 算力、数据、专家时间有限, 投给 A 的资源就少投给 B。真正更危险的, 是横向能力之间发生冲突, 也就是负迁移。

负迁移

负迁移指训练让某一项行为变好, 同时损伤另一项原本有价值的能力。视频里的例子是显式指令遵循与隐式意图理解: 模型如果被强烈训练成“完全按字面执行”, 用户文件名里有拼写错误时, 它可能真的去改那个错文件; 人类助手通常会察觉这里可能是 typo, 然后先澄清或选择更合理的目标。

显式指令遵循很重要。用户说“只改这个文件”“不要动其他文件”, 模型必须严格执行。隐式意图理解也很重要。用户说错文件名、漏掉约束、写出含糊需求时, 一个有用的助手需要推断真实意图, 必要时提问。问题出在两者都像方向盘: 一个要求沿字面直行, 一个要求根据路况微调。训练若过度偏向任何一边, 都会带来产品风险。

可以把冲突写成一个多目标优化问题:

$$J_{\text{instr}}(\theta) = \lambda_{\text{explicit}} S_{\text{explicit}}(\theta) + \lambda_{\text{implicit}} S_{\text{implicit}}(\theta) - \lambda_{\text{risk}} C_{\text{mistake}}(\theta)$$

- $J_{\text{instr}}(\theta)$: 本节用于说明指令遵循权衡的局部目标, 不等于第 5 节的 RL 训练目标。
- $S_{\text{explicit}}(\theta)$: 显式指令遵循得分, 衡量是否按用户字面要求执行。
- $S_{\text{implicit}}(\theta)$: 隐式意图理解得分, 衡量是否能处理 typo、歧义和上下文暗示。

- $C_{\text{mistake}}(\theta)$: 错误执行成本，例如误改文件、误用工具、误解任务。
- $\lambda_{\text{explicit}}, \lambda_{\text{implicit}}, \lambda_{\text{risk}}$: 不同目标的权重。

这个公式的意义在于说明：听话程度越高未必带来单调增益，猜测能力越强也未必带来单调增益。高质量助手需要在二者之间学会切换：约束明确且风险高时严格遵循；输入明显有误且风险可控时提醒或澄清；需要执行不可逆操作时停下来确认。

7.5 法律、医疗和经济任务为什么仍可训练

Dubois 对更广泛经济领域的判断偏乐观：模型能力本身没有明显理由只能擅长代码、数学、网络安全，不能达到法律、医疗和其他专业领域的高水平。真正的阻力来自三件事。

第一，专家反馈稀缺。代码模型的训练者通常也是重度用户，能每天感知产品质量。法律和医疗需要专业资格、行业语境、责任边界，模型团队很难只靠内部直觉判断输出好坏。

第二，可验证奖励不均匀。网络安全容易判断“漏洞是否真实存在”，代码可以运行测试，数学可以检查答案。法律建议是否稳妥、医疗判断是否合适、经济报告是否抓住关键变量，往往需要多证据、多专家、多时间尺度的评价。

第三，真实任务需要外部资源。模型不能只在 prompt 内推理，还要检索资料、引用证据、比较来源、识别数据口径。这些能力和领域知识互相缠绕，训练难度高于封闭题。

为什么仍然可训练

法律、医疗、经济任务并没有神秘壁垒。只要能构造高质量任务、专家反馈、可靠评测、可控工具环境，RL 和后训练仍可逐步优化这些领域。难点在工程组织和评测闭环，不在“模型天生无法理解这些行业”。

本章要保留的核心判断是：跨领域进步会发生，但速度不均匀。算法可以迁移，能力可以横向扩散；数据、专家、评测与奖励决定扩散的落地速度。把模型从竞赛题带进经济世界，真正要补的是开放任务里的校准、检索、拒答、澄清和工具化 workflow。

7.6 本章小结

本章把泛化拆成三层：算法能否搬家，能力能否跨任务扩散，领域能否形成训练闭环。GDPval 在视频中代表经济任务评测的转向，说明模型进步开始进入真实生产率问题。SFT 可能把“看似标准的人类答案”复制成幻觉行为，RL 在奖励可验证时能压低编造概率。负迁移提醒读者，横向能力之间会互相拉扯，尤其是显式指令遵循和隐式意图理解。法律、医疗、经济任务可以训练，但需要专家、数据、奖励和评测共同到位。

8 评测成为瓶颈：开放任务、专家稀缺与 Model as Judge

8.1 模型越强，评测越难

Yann Dubois 对 eval 的第一句话很关键：模型越强，评测越难。原因并不玄学。弱模型做的是边界清楚的小任务，评测者可以提前列出答案；强模型开始处理“给我做一个网站”“帮我分析

一个行业”“找出这套实现中的根因”这类开放任务，输出空间一下变大。过去的评测像批改选择题，未来的评测更像评审一个项目交付物。

闭式任务可以用准确率表达。比如一组题目有唯一标准答案，模型给出 $\hat{y}_i$ ，标准答案是 y_i ，那么：

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i]$$

- N ：评测样本总数。
- i ：第 i 个样本的索引。
- $\hat{y}_i$ ：模型在第 i 个样本上的输出。
- y_i ：第 i 个样本的标准答案。
- $\mathbf{1}[\hat{y}_i = y_i]$ ：指示函数；输出等于标准答案时为 1，否则为 0。
- Acc ：准确率或通过率，表示模型答对的样本比例。

这个公式适合数学题、单元测试、已知 bug 列表、可执行代码检查等任务。它的优点是便宜、稳定、可重复；缺点也明显：一旦任务没有唯一答案，这个公式就开始失灵。一个网站可以有多种合理实现，一个分析报告可以有不同但都站得住的结构，一个修复方案也可能在安全性、性能、可维护性之间做不同取舍。

评测从“判答案”变成“判交付”

前沿模型的评测瓶颈来自任务形态变化。封闭任务要求模型命中标准答案；开放任务要求模型交付可用结果。前者可以数对错，后者需要判断质量、约束、审美、风险、证据和上下文适配。

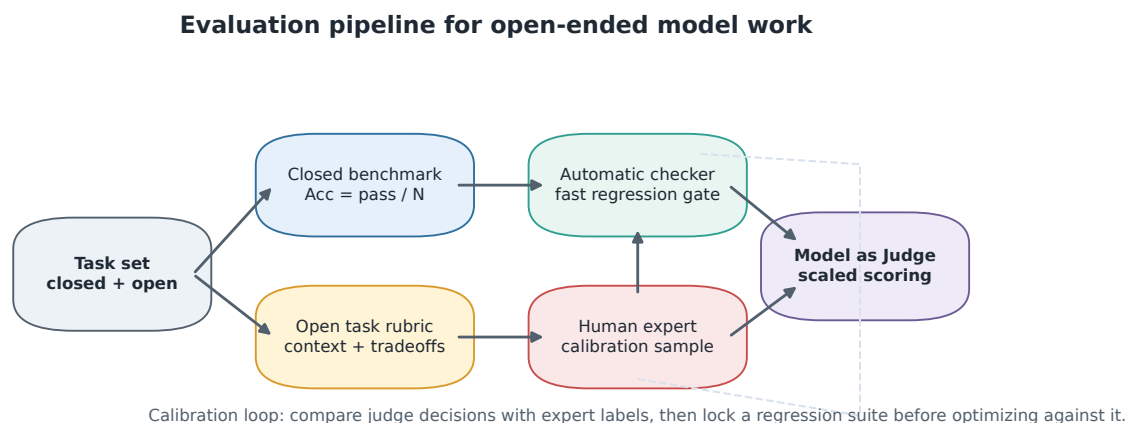


图 16: 开放任务评测流水线：闭式基准评测可以用准确率或通过率快速回归，开放任务需要 rubric、人类专家校准和模型裁判（Model as Judge）的可审计扩展。

8.2 开放任务为什么没有唯一标准答案

Dubois 举的网站例子很好：用户说“帮我建一个做 X 的网站”，很难预先写出唯一最优答案。页面结构、交互方式、技术栈、视觉风格、可访问性、性能优化，都可能有多种合理路线。评测者不能只问“是否等于答案”，必须问“是否满足目标、是否鲁棒、是否适合用户场景”。

开放任务常见的难点有四类。第一，目标不完整。用户经常只说结果，不说约束。第二，答案空间很大。同一目标可以有多个实现路径。第三，质量维度冲突。更快的方案可能牺牲可维护性，更华丽的界面可能牺牲可访问性。第四，评测本身会改变训练。Dubois 指出，每次构建一个 eval，几乎也在构建一类训练数据；模型很快可以在类似数据上变强，使旧 eval 过时。

因此，开放任务更适合用 judge score 表达：

$$\text{Score} = \frac{1}{N} \sum_{i=1}^N S_j(x_i, y_i)$$

- N ：开放任务样本总数。
- x_i ：第 i 个任务输入，例如“构建一个网站”或“分析一份合同”。
- y_i ：模型对任务 x_i 的输出或交付物。
- j ：负责评分的裁判者，可以是人类专家、裁判模型，或二者组合。
- $S_j(x_i, y_i)$ ：judge j 对输出 y_i 在任务 x_i 下的质量评分。
- Score：开放任务平均评分，用来近似衡量交付质量。

这个公式比准确率更贴近真实工作，但也更脆。评分标准写得粗，模型会学会迎合表面特征；评分标准写得细，成本会上升，专家也更难保持一致。开放任务的 eval 本质上是一套小型制度：它必须定义目标、证据、权重、边界和复核机制。

8.3 人类专家稀缺：当模型超过多数评审者

评测难的第二层来自专家稀缺。模型在某些轴上已经超过多数人类，能评它的人越来越少。数学、代码、网络安全、复杂研究摘要、法律条款审查，都可能出现类似局面：普通标注员看不出模型错在哪里，领域专家能看出来，但数量少、成本高、时间碎片化。

这会造成一个现实瓶颈：训练模型的人想推进能力，首先要知道模型哪里坏了；若没有高质量 eval，就无法稳定衡量改进。Dubois 还提到一个文化问题：很多人天然更愿意“训练模型”，觉得那才是在提升模型；找问题、做 eval、量化改进常被低估。实话说，这种低估会直接拖慢前沿进展。没有仪表盘，踩油门只会更危险。

评测工作被低估的代价

评测若只被看成发布前的打分表或论文里的附录，就会被低估。对前沿模型来说，eval 是发现缺陷、制造训练数据、比较版本、约束奖励函数的基础设施。若组织把 eval 当成二线工作，训练团队会在噪声里优化，最后得到的只是更会通过旧题的模型。

专家稀缺还会带来一致性问题。两个顶级专家可能都能评判答案，却未必使用同一套偏好权重。法律里有人更重视风险规避，有人更重视商业可执行；医疗里有人更重视敏感性，有人更重

视减少不必要干预。开放任务 eval 必须把这种分歧显式化，不能假装存在一个天然唯一的“专家答案”。

8.4 Model as Judge: 用模型评测模型的机会与风险

Model as Judge 的吸引力很直接：更强模型可以当老师，也可以当评审者。Dubois 把它称为能力飞轮的一部分。模型变强后，可以帮助构造训练数据、比较答案、发现弱点，再推动下一代模型变强。对开放任务来说，这几乎是绕不开的方向，因为完全依赖人类专家会太慢。

可以用裁判-专家一致率 (judge-human agreement) 来刻画裁判模型的可靠性：

$$q = P(J(x, y) = H(x, y))$$

- q ：裁判模型与人类专家判断一致的概率。
- $J(x, y)$ ：裁判模型对任务 x 和输出 y 的判断。
- $H(x, y)$ ：人类专家对同一任务和输出的判断。
- $P(\cdot)$ ：概率，表示在任务分布上发生某种事件的比例。

若 q 高，裁判模型可以扩大评测规模，降低专家负担；若 q 低，它会把偏见和漏洞放大。更麻烦的是，训练和评测使用相近数据时，eval 会很快老化。模型可能学会满足裁判者的偏好，却没有真正提升真实效用。这和考试刷题类似：分数上升可能表示能力提高，也可能表示题库被吃透。

Model as Judge 的主要风险

裁判模型的风险不只是“会打错分”。更大的问题是反馈回路：被评模型学习裁判者的偏好，裁判者又由相近模型或相近数据训练，系统可能形成自我强化的盲区。开放任务越依赖模型裁判，越需要人类抽检、对抗样本、跨模型裁判、真实用户结果和长期回溯。

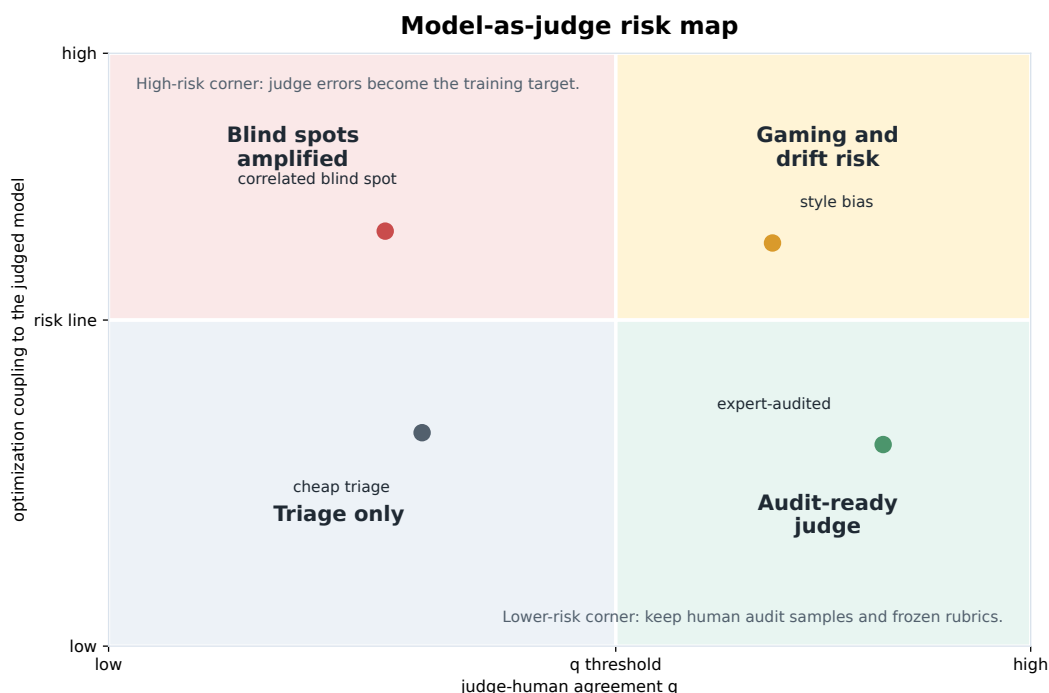


图 17: Model as Judge 风险图：横轴是裁判-专家一致率 q ，纵轴是优化目标与裁判模型的耦合强度；高耦合会放大共同盲点和风格偏差。

不过，机会同样很大。一个设计良好的 Model as Judge 系统可以完成三件事：第一，快速筛掉明显差的输出，把专家时间留给边界案例；第二，为 RL 提供可扩展奖励信号；第三，持续发现新 failure mode，让 eval 动态更新。它像机场安检里的多层筛查：机器先过一遍，人类处理高风险和难判断样本，事后还要抽查系统有没有漏判。

8.5 本章小结

本章把 eval 放回前沿模型训练的核心位置。闭式任务可以用准确率衡量，开放任务需要裁判分数和多维质量判断。模型越强，任务越开放，评测越依赖稀缺专家，也越容易被组织文化低估。模型裁判提供了规模化评测与训练数据生成的机会，同时带来反馈回路、偏好投机、eval 老化和盲区放大的风险。下一阶段的模型进步，很大一部分会取决于评测系统能否跟上模型能力。

9 未来图景：持续学习、Agent Harness 与最后一公里

9.1 连续进步与局部跳跃：为什么编码先有体感

谈到未来 12 到 24 个月，Yann Dubois 的判断很克制：底层能力大多是连续推进的，人的体感却会出现局部跳跃。这个说法延续了开篇的主线。水位一直在涨，闸门只有在某个高度被顶开时才会让岸上的人突然感到“变天了”。编码和软件工程先出现强烈体感，原因并不神秘：它同时具备高频使用、即时反馈、可运行验证、训练者亲自使用这几项条件。

编码任务对模型进步非常敏感。一个代码 agent 每隔几分钟都可能犯错，单步错误率下降一点，长任务成功率就会放大；代码又能通过测试、编译、静态分析、运行结果来验证，反馈闭环比法律、医疗、管理咨询更短。更关键的是，训练和使用这些模型的人本身大量写代码，产品问题可

以被内部团队直接感知。软件工程因此像一条已经铺好传感器的赛道：车一快，速度变化马上被看见。

Dubois 说，其他垂直领域也会陆续感到类似变化。这里要注意“局部跳跃”的含义。它并不意味着所有行业同一天同步跃迁；它指某个行业、某类任务、某种工作流在可靠性、评测、工具、权限和数据同时过线之后，用户突然从“可以玩”转向“可以交付”。从远处看，整体曲线仍然平滑；贴近某个岗位看，体验会像台阶。

局部跳跃的来源

所谓局部跳跃，通常来自连续能力跨过具体工作流的阈值。编码先有体感，因为它把高频用户、可验证任务、工具调用、训练反馈和内部需求放在同一个闭环里。其他领域也会发生类似跃迁，但速度取决于该领域能否形成同样紧的反馈系统。

9.2 持续学习：时间-效用曲线与企业记忆

Dubois 对持续学习非常兴奋，同时明确说“还没有真正解决”。他还补充，这个问题在 ChatGPT 出现后就显得很自然，很多人曾以为几个月内会被解决；三年后仍未完成，说明瓶颈同时来自权限、隐私、产品形态、资源投入和组织边界。他给出的图很值得保留：横轴是时间，纵轴是模型给用户提供的效用。今天的强模型被放进一家公司时，第一天可能比许多新员工更有用，因为它已经带着广泛知识、代码能力、写作能力和工具使用能力进入现场。问题在后面：模型的效用曲线往往比较平，人的效用曲线会上升。人会学公司黑话、权限边界、历史决策、客户偏好、代码库惯例、老板的真实要求；模型如果只靠一次次会话，很难把这些经验稳定沉淀下来。

这里还应保留 Yann 的一个重要停顿：当主持人追问“根本困难是什么”时，他没有给出包装得很完整的答案，只是坦率说自己并不完全知道为什么行业花了这么久还没有解决。企业场景里的权限、隐私、跨用户共享当然很难；但他同时强调，即使只看单个用户，记忆和个性化也还没有到理想状态。这说明持续学习的瓶颈不能被简化成“把资料存起来再检索”，它仍是一个产品、训练、隐私和长期反馈共同缠绕的开放问题。

可以把持续学习的价值写成一条用户效用曲线。先看直觉：某个系统在一天、一个月、一年里到底有没有价值，不只看第一天的分数，也看它随时间积累了多少有效贡献。

$$\text{AUC}_U = \int_0^T u(t) dt$$

- t ：系统进入用户或企业环境之后的时间。
- T ：观察窗口，例如一个项目周期、一个季度或一年。
- $u(t)$ ：时刻 t 的用户效用，可以理解为模型在该环境中节省的时间、降低的错误、提高的产出质量或带来的业务价值。
- AUC_U ：效用-时间曲线下面积，表示整个时间窗口内的累计价值。

这个公式的重点在积分。一个系统如果 $u(0)$ 很高，但之后长期不变，它的累计价值会被会学习的人类同事追上。持续学习要改变的是曲线形状：让 $u(t)$ 随着环境暴露、反馈积累和流程嵌入单调上升，至少在相当长的一段时间里保持正斜率。

进一步写成增量形式，会更清楚地看到“记忆”为什么重要：

$$u(t + \Delta t) = u(t) + \Delta u_{\text{memory}} + \Delta u_{\text{workflow}} + \Delta u_{\text{feedback}} - \Delta u_{\text{drift}}$$

- Δt : 一次学习或一个工作周期的时间间隔。
- Δu_{memory} : 来自用户偏好、项目背景、企业知识沉淀的效用增量。
- $\Delta u_{\text{workflow}}$: 来自对工具链、审批流、代码库、客户系统熟悉之后的效用增量。
- $\Delta u_{\text{feedback}}$: 来自纠错、评价、人工复核和成功案例的效用增量。
- Δu_{drift} : 记忆过期、错误沉淀、权限变化、组织结构变化带来的效用损失。

企业记忆的难点就在这里。记忆不是把所有聊天记录塞进向量库。它更像公司里的“制度化经验”：什么信息可以被谁看到，什么偏好只属于个人，什么结论已经过期，什么流程具有法律风险，什么知识可以跨团队复用。Dubois 特别提到权限和隐私问题。企业场景里，一个销售的客户信息、一个法务的合同草稿、一个工程团队的安全漏洞、一个财务团队的预算计划，不能因为“模型要学习”就混在一起。持续学习越深入，访问控制、审计、遗忘机制、数据血缘和组织边界越重要。

把记忆当成搜索缓存会低估问题

企业记忆包含事实、偏好、权限、时间、来源和责任。只做检索增强，通常只能解决“找到材料”；持续学习还要解决“材料能不能用、对谁可见、什么时候过期、使用后谁负责”。这也是单用户记忆都尚未完全成熟，企业级记忆更慢的原因。

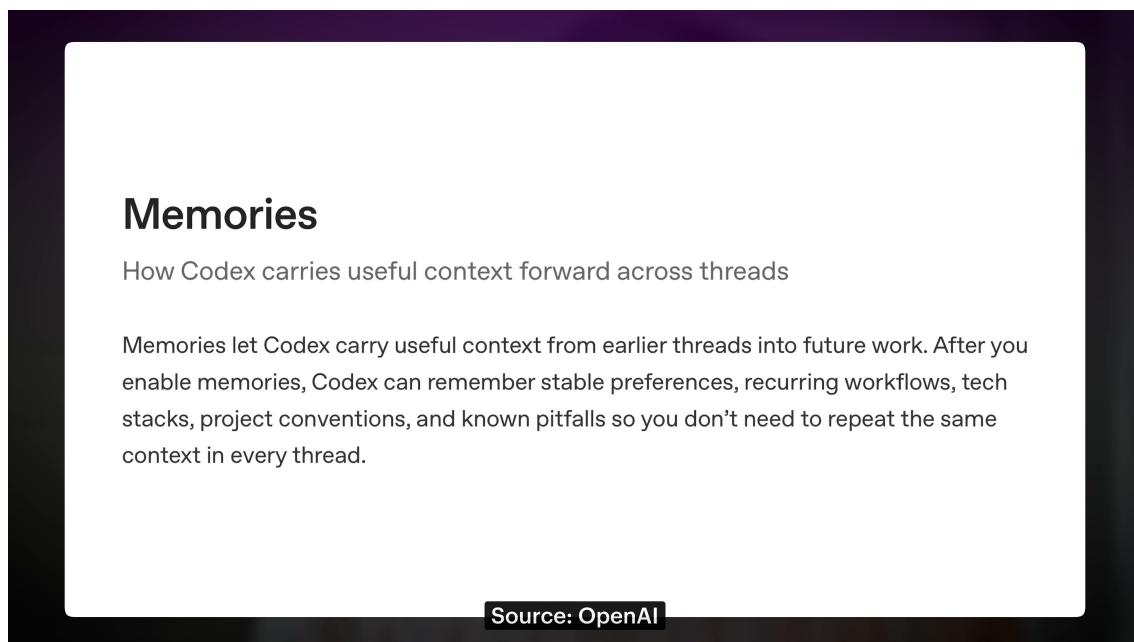


图 18: 视频中的 Memories topic frame: 该帧明确把记忆能力解释为把稳定偏好、重复工作流、技术栈、项目约定和已知坑点带入未来线程，适合放在第 9 节持续学习与企业记忆讨论前。⁴

⁴视频帧来源时间区间：01:06:00–01:06:30。

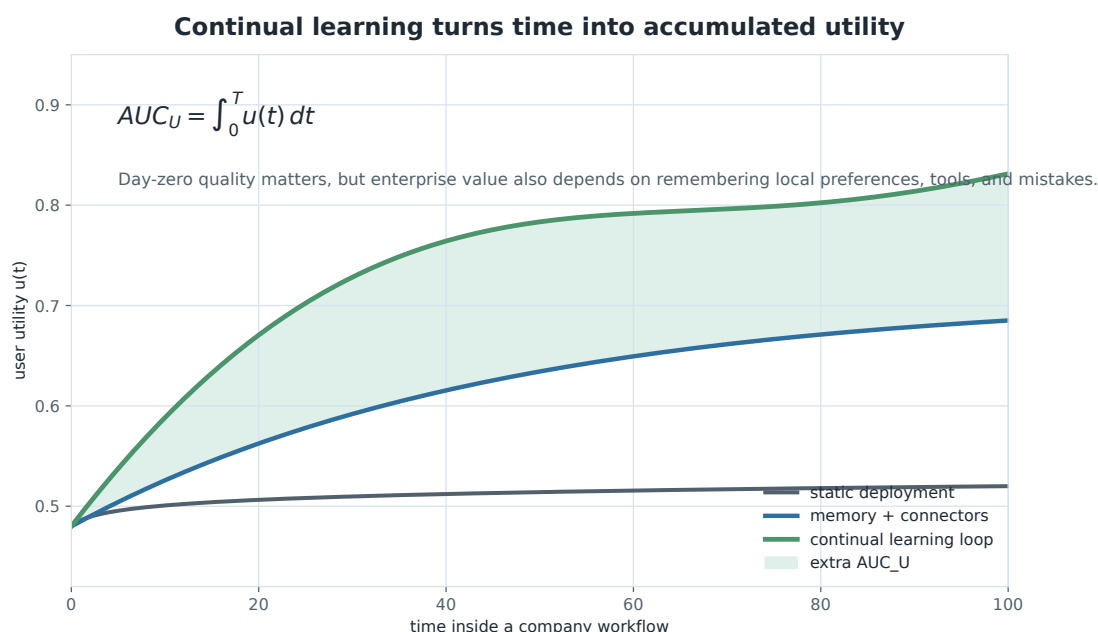


图 19: 持续学习效用曲线: $u(t)$ 表示系统在企业 workflow 中的用户效用, $AUC_U = \int_0^T u(t) dt$ 强调记忆、连接器和持续学习如何把时间转化为累计价值。

9.3 Agent Harness: 短期增益、长期重调与领域适配

主持人把问题推进到一个现实张力: 模型越来越强, 围绕模型搭建的 RAG、Agent Harness (智能体外部脚手架)、工作流编排, 会不会被基础模型吃掉? Dubois 的回答很实在。今天的 Agent Harness 确实能显著提高模型能力, 尤其在具体垂直目标上, 把可靠性从 80% 推到 85% 这类增益很有价值。代价是它需要随着模型更新不断重调。

Agent Harness 可以理解为模型外部的脚手架: 工具、路由、记忆、权限、连接器、计划器、浏览器、代码执行环境、复核规则、失败重试策略、领域模板都属于其中。它像赛车的维修站和赛道规则。再好的发动机, 也需要轮胎、燃料、刹车、导航和安全边界; 发动机升级以后, 整套调校也要跟着改。

Agent Harness 的组成

一个实用的 Agent Harness 往往包括六层: 任务路由, 决定该用哪个模型或模式; 上下文管理, 决定带入哪些资料 and 记忆; 工具接口, 连接浏览器、代码、数据库、文档和业务系统; 权限系统, 限定模型能看什么、能改什么; 执行监控, 处理重试、回滚、人工确认; 评测闭环, 把失败案例变成后续改进材料。

Dubois 对“通用且长期稳定的 Agent Harness”并不乐观。理由很直接: 基础模型的水平还在快速变化, 今天用外部逻辑补的短板, 明天可能被模型本身补上; 今天适合的提示词、工具顺序、失败处理策略, 下一代模型可能需要不同配置。于是, Agent Harness 的合理定位是面向具体目标的短中期产品工程, 目标不在于冻结一套永恒外壳。

这并不削弱 Agent Harness 的价值。Dubois 甚至说, 如果冻结现有模型, 围绕它们认真做 Agent Harness, 并把训练也放进优秀 Agent Harness 环境里, 很多领域已经会让人强烈感到 AGI 式能力。问题在于模型不会冻结。产品团队要接受一个硬事实: Agent Harness 是会折旧的资产。

它带来当下收益，也会产生维护负担；越贴近具体业务，越能创造价值，迁移性也越有限。

Harness 的战略判断

具体领域、明确目标、可测收益的 Agent Harness 值得做；试图跨所有行业、跨多代模型保持稳定的通用 Agent Harness 风险很高。更好的问题应当换成：“这套外部脚手架在下次模型升级前能产生多少确定价值，以及升级后重调成本是否可控？”

9.4 最后一公里：权限、连接器、流程嵌入与创业空间

最后一公里是 Dubois 给创业者和应用团队留下的最重要判断。他不同意把瓶颈简单归结为“原始智能”。很多时候，模型已经足够聪明，真正卡住价值的是它进不了现场：没有正确权限，连不上关键系统，不懂审批链，不能安全改数据，不知道什么时候要找人确认，也没有被嵌入用户每天真实使用的流程。

所谓最后一公里，是把通用能力送进具体工作。它包括四类工作。第一是权限，让模型知道自己能看什么、能做什么、哪些动作必须人工确认。第二是连接器，把 CRM、ERP、代码仓库、票据系统、文档库、邮件、日历、数据库接入进来。第三是流程嵌入，让模型不只是回答问题，还能在用户已经工作的地方完成步骤。第四是领域适配，把行业语言、合规边界、成功标准和评测闭环变成产品的一部分。

Last-mile stack: where raw capability becomes workflow value

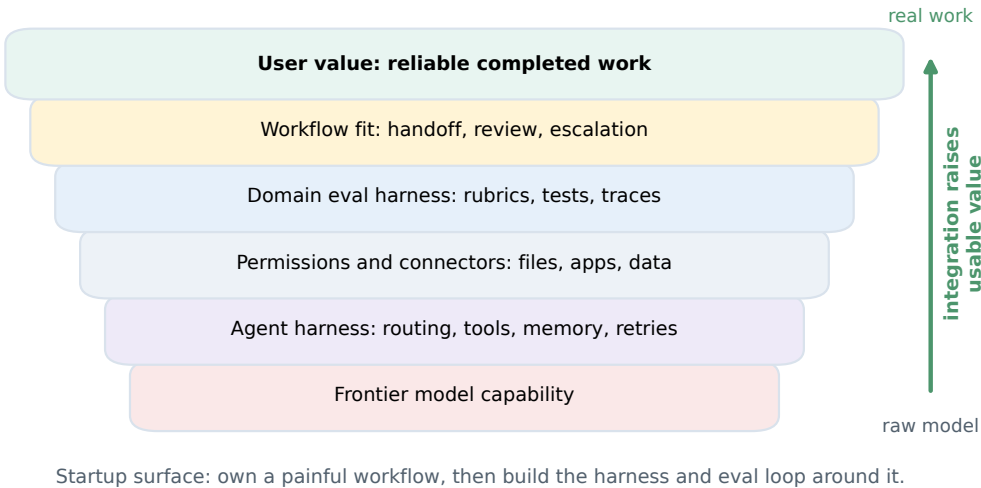


图 20: 最后一公里产品栈：从前沿模型能力到 Agent Harness、权限连接器、领域评测、 workflow 嵌入和用户价值，逐层说明为什么创业机会常出现在模型外壳与业务流程交界处。

这也是创业空间（startup space）的来源。大模型公司会优先推进横向能力：更强推理、更好工具使用、更低延迟、更稳可靠性、更通用的记忆和评测。垂直公司可以把注意力放在某一类工作流上，把现有能力榨干。比如客服自动化要接入工单、知识库、历史对话和升级规则；投行业务要接入模型、表格、审计痕迹和审批；法务产品要处理权限、版本、引用来源和责任边界。这些问

题不会因为模型更聪明就自动消失。主持人在这里列举的 Tau2-bench Telecom、FinanceAgent 和 OfficeQA Pro 等例子，说明模型能力已经逐步进入客服、电信、投行建模和办公问答等更接近真实工作的任务；其中访谈原文提到 FinanceAgent 可自动化 88.5% 的内部投行建模任务，OfficeQA Pro 达到 51.1%。这些数字的作用在于把最后一公里问题放到真实业务流程的进展背景下，不能被理解成所有行业都已解决。

当然，创业者也要清醒。最后一公里不能只套一层 UI，也不能只把几个 API 串起来。护城河来自真实分发、深流程、专有反馈、合规经验、数据闭环和对失败成本的理解。若某个产品只是在通用模型外面包一层薄壳，并且没有独特数据、独特工作流或独特权限位置，它很容易被平台功能吸收。若它掌握的是行业现场的复杂边界，模型进步反而会提高它能交付的价值上限。

创业空间的硬边界

“模型会越来越强”对应用层既是机会，也是压力。机会在于可用能力持续抬升；压力在于浅层包装会快速贬值。真正值得押注的最后一公里，通常藏在权限、连接器、流程责任、行业评测和用户分发里。

9.5 本章小结

本章把未来图景落到三个工程问题上。第一，AI 进步大体连续，但会在具体行业形成局部跳跃；编码先有体感，因为它有最短的反馈闭环和最密集的高质量用户。第二，持续学习的关键指标是效用-时间曲线的面积 AUC_U ，模型需要从高 $u(0)$ 走向随时间上升的 $u(t)$ ，这要求企业记忆同时处理知识、权限、隐私、时效和责任。第三，Agent Harness 和最后一公里仍然重要；它们把通用模型能力变成具体工作流价值，也会随着基础模型升级不断重调。对创业者来说，空间还很大，但浅层封装会很脆，深流程和真实反馈才是更硬的资产。

10 总结与延展：后训练时代的能力、可靠性与产品边界

10.1 全片主线压缩：能力、可靠性、评测、产品

这场对话的主线可以压缩成一句话：后训练时代的前沿模型进步，已经从“模型会不会”转向“系统能不能稳定交付”。Yann Dubois 反复强调的几个点彼此扣在一起。预训练给模型广泛先验，后训练把潜在能力变成可交互行为，强化学习把可验证奖励扩展到真实用户效用，测试时计算 (test-time compute) 提供性能与延迟之间的旋钮，评测和模型裁判 (Model as Judge) 决定系统能否知道自己变好了多少，最后一公里决定这些能力能否进入真实工作流。

开篇的可靠性阈值解释了为什么进步突然有体感。模型能力并非凭空跳变，体感跳变来自多步 agent 工作的错误率终于降到可用区间。中段的后训练讨论解释了能力如何被塑形：SFT 复制人类示范，RL 通过奖励信号推动模型超越示范并压低幻觉、错误工具调用、低效推理等行为。评测章节说明了瓶颈为什么转向 eval：任务越开放，标准答案越少，专家越稀缺，模型裁判越重要也越危险。最后的结尾讨论则把未来落到持续学习、Agent Harness 和应用创业空间上。

Dubois 的实质性收束有五个判断。第一，长期看能力大多连续进步；第二，局部体感会出现不连续感，编码已经发生，其他垂直领域还会陆续发生；第三，持续学习仍是明显缺口，模型第一天很强，却还不会像新人一样持续吸收公司知识；第四，Agent Harness 现在能创造大量增益，但需要随着基础模型变化不断重调；第五，创业者仍有最后一公里空间，因为真实瓶颈经常是权限、

连接器、流程嵌入和领域交付。这组判断也回应了主持人关于 accelerating loops 和 AI building AI 的问题：能力飞轮已经存在，短期最明确的瓶颈落在持续学习、评测和工作流嵌入上。

全片的核心压缩

本片讨论的核心，不止是单一模型参数变大，更是一套闭环系统成熟：可靠性降低长任务失败率，后训练把能力调成可用行为，测试时计算让性能可调，评测让改进可见，持续学习和最后一公里让模型进入长期工作环境。任何一环短板明显，用户感到的生产力都会被压住。

10.2 从公式到直觉：四条曲线看懂这次进步

第一条是可靠性曲线。Agent 工作像一串连续动作，单步错误率 p_i 很小，也会在长任务里累积。前文可写成：

$$P_{\text{success}}(T) = \prod_{i=1}^T (1 - p_i) \approx e^{-\sum_i p_i}$$

- T ：一个长任务中的工作步数或时间片数量。
- p_i ：第 i 步发生关键错误的概率。
- $P_{\text{success}}(T)$ ：任务从头到尾不发生致命错误的概率。
- $\prod$ ：连乘，表示每一步都不出错才能完成任务。
- $e^{-\sum_i p_i}$ ：当每步错误率较小时的近似形式，用来显示错误会随步数累积。

这条曲线解释了阈值体感。单步错误率从 3% 降到 2%，在十步任务里只是变好一点；在百步 agent 工作里，成功率差异会被放大。编码 agent、电脑使用、研究助理和企业流程自动化都受这条曲线控制。

第二条是测试时计算曲线。模型思考得更久，通常会更准，但收益会递减。可以用一个示意函数表达：

$$A(k) = A_{\text{max}} - c(k + k_0)^{-\alpha}$$

- k ：测试时计算预算，例如思考 token、工具调用次数或搜索深度。
- $A(k)$ ：在预算 k 下的准确率、任务分数或用户效用。
- A_{max} ：该任务和模型系统在当前条件下接近的上限。
- c ：距离上限的缩放系数。
- k_0 ：避免 $k = 0$ 附近形式失真的平滑项。
- α ：收益递减速度，越大表示增加计算带来的改善越快接近平台。

这条曲线解释 Thinking 与 Pro 的关系，也解释效率为什么是研究和工程共同问题。研究把曲线向左推，让同样表现需要更少思考；推理系统把 token 预算映射成真实延迟；产品再决定用户愿意为多高质量等多久。

第三条是 eval/judge 曲线。封闭任务可以用准确率，开放任务需要裁判分数和人类一致性。最简洁的开放任务形式是：

$$\text{Score} = \frac{1}{N} \sum_{i=1}^N S_j(x_i, y_i), \quad q = P(J(x, y) = H(x, y))$$

- N ：评测样本数量。
- x_i ：第 i 个开放任务输入。
- y_i ：模型对任务 x_i 的输出或交付物。
- $S_j(x_i, y_i)$ ：裁判者 j 对输出质量的评分。
- $J(x, y)$ ：裁判模型对任务和输出的判断。
- $H(x, y)$ ：人类专家对同一任务和输出的判断。
- q ：裁判模型与人类专家一致的概率。

这条曲线解释了 eval 为什么成为瓶颈。模型越强，越常处理没有唯一答案的任务；模型裁判越强，越能扩展评测规模；judge 若与专家偏离，就会把偏差写进训练闭环。

第四条是持续学习效用曲线。Dubois 在 closing discussion 中强调的正是这条曲线：

$$\text{AUC}_U = \int_0^T u(t) dt$$

- $u(t)$ ：模型在时间 t 对用户或企业产生的效用。
- T ：观察期长度。
- AUC_U ：累计效用，衡量系统在整个工作期内创造了多少价值。

这条曲线解释了持续学习为什么关键。今天的模型 $u(0)$ 很高，像一位入职第一天就懂很多通识的新同事；它如果不能学习企业知识、流程、偏好和失败案例，曲线会变平。人类新员工的起点可能更低，但斜率更高。真正成熟的企业 AI 要把高起点和正斜率结合起来。

Four curves that compress the whole lecture



图 21: 四条曲线总览：长任务可靠性、测试时计算缩放、评测信号分叉和部署后的效用积累共同压缩全片主线，适合作为第 10 节综合图。

四条曲线之间的关系

可靠性曲线决定 agent 能否跑长任务；测试时计算曲线决定质量、成本和延迟如何交换；评测/裁判曲线决定改进能否被测量并转化为训练信号；持续学习效用曲线决定模型进入真实组织之后能否越用越值钱。四条曲线叠在一起，才构成用户感到“AI 真的能干活了”的系统条件。

10.3 对研究者、产品团队和创业者的具体启示

对研究者来说，后训练已经成为能力显化的核心现场。单纯追求更高基准评测（benchmark）分数不够，研究问题要贴近真实效用：如何把 verifiable reward 推向开放任务，如何让 RL 在长 rollout 中做稳定信用分配，如何构建不易老化的 eval，如何让裁判模型与专家判断保持一致，如何让记忆在隐私和权限边界内持续更新。持续学习尤其值得投入，因为它连接模型能力、数据治理、用户反馈和组织知识，短板非常明显。

对产品团队来说，最重要的是把“模型能力”翻译成“工作流可靠性”。一个功能能不能上线，不只取决于模型单次回答好不好，还取决于失败时有没有复核，权限是否可控，日志是否可审计，连接器是否稳定，用户是否能在原工作流里完成任务。产品经理要盯住三类指标：长任务成功率、端到端延迟与成本、真实用户场景下的纠错闭环。模型版本升级后，提示词、工具顺序、Agent Harness 策略和评测集都要重新校准。

对创业者来说，最后一公里仍有空间，但空间集中在硬问题上。值得做的垂直应用通常拥有至少一个优势：独特分发渠道，深度行业流程，难以复制的数据闭环，关键系统权限位置，专家复核网络，或对失败成本的细致理解。只做薄封装会面临平台吸收和模型升级的双重压力。做深流

程则会受益于模型进步，因为每一代模型都能把同一套权限、连接器和反馈系统变得更有产出。

不要把“模型会变强”误读成“应用会消失”

模型变强会吃掉一部分浅层功能，也会打开更复杂的工作流。应用层的边界会移动，价值不会平均消失。越靠近真实权限、真实数据、真实责任、真实用户分发的产品，越可能把横向模型进步转化成纵向业务价值。

10.4 开放问题：持续学习、隐私边界与真实世界奖励

第一个开放问题是持续学习怎样落地。单用户记忆已经有帮助，但距离“越用越懂我”仍有距离；企业记忆更复杂，因为它牵涉权限、组织结构、知识过期、跨用户共享和审计责任。一个可行系统可能需要把短期上下文、长期偏好、企业知识库、任务反馈和显式授权分层管理，不能把所有信息混成一团。

第二个开放问题是真实世界奖励怎样构造。数学、代码、网络安全较容易自动验证，法律、医疗、金融、管理任务常常没有单一正确答案。未来的 RL 若要进入这些领域，需要更强的 judge、更好的专家抽样、更细的评分规约、更长周期的结果回收。奖励函数越接近真实价值，训练越有效；奖励函数越粗糙，模型越会学会投机。

第三个开放问题是 Agent Harness 与基础模型的边界如何移动。当前 Agent Harness 能显著提高可靠性和领域适配，但模型升级会改变外部脚手架的最佳设计。未来的产品架构更可能是一层可重调、可观测、可替换的控制层：模型负责越来越多通用推理，Agent Harness 保留权限、工具、流程、审计和领域策略。

第四个开放问题是评测如何避免自我污染。模型裁判 (Model as Judge) 会扩大评测规模，也可能把裁判偏好变成训练捷径。开放任务的 eval 需要动态更新、对抗样本、跨模型裁判、人类专家抽检和真实用户结果校验。说得直白一点，如果评测系统跟不上，模型团队会在越来越漂亮的分数里失去方向。

10.5 本章小结

本章把全片内容收束为四条曲线和三类实践启示。可靠性曲线解释了 agent 长任务为什么有阈值；测试时计算曲线解释了 Thinking、Pro、效率和延迟的取舍；评测/裁判曲线解释了开放任务为什么让评测成为一线瓶颈；持续学习效用曲线解释了企业 AI 为什么必须越用越有价值。Dubois 的结尾讨论给出的方向很明确：进步会持续，局部跃迁会扩散，持续学习仍有缺口，Agent Harness 会在具体领域继续创造短期增益，最后一公里仍是产品和创业的主战场。后训练时代的边界，不只在模型内部，也在评测、权限、连接器、记忆和真实工作流里。